

 iamSaikrishna143 / Last_Believe



[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security](#)

 [main](#) **Last_Believe / jsinterview.js** 





iamSaikrishna143 all update

ce579bf · 9 hours ago



4074 lines (2710 loc) · 129 KB

```
1 // 1. What is the difference between 'Pass by Value' and 'Pass by Reference'?
2 // Ans:
3 // In Pass by Value, a copy of the original value is created and passed to the
4 // function. Any changes made to the value within the function do not affect
5 // the original value outside the function.
6
7 function changeValue(x) {
8     x = 10; // changing the value of x
9     console.log("Inside function: x =", x);
10 }
11 let x = 5;
12 console.log("Before calling function: x =", x);
13 changeValue(x);
14 console.log("After calling function: x =", x);
15 // output:
16 // Before calling function: x = 5
17 // Inside function: x = 10
18 // After calling function: x = 5
19 // Pass by Reference
20 // In Pass by Reference, a reference to the original value is passed to the function. Ar
21 // to the value within the function affect the original value outside the function.
22
23 function changeValue(obj) {
24     obj.value = 10; // changing the value of the object
25     console.log("Inside function: obj.value =", obj.value);
26 }
27 let obj = { value: 5 };
28 console.log("Before calling function: obj.value =", obj.value);
29 changeValue(obj);
30 console.log("After calling function: obj.value =", obj.value);
31 // output:
32 // Before calling function: obj.value = 5
33 // Inside function: obj.value = 10
34 // After calling function: obj.value = 10
35 // -----
36 // 2. What is the difference between map and filter ?
```

```

37 // The map() method creates a new array with the results of applying a provided
38 // function to every element in the original array. It returns a new array with
39 // the same length as the original array, where each element is the result of the
40 // transformation function.
41 // Syntax
42 // arr.map(callbackFunction(currentValue, index, array))
43 const numbers = [1, 2, 3, 4, 5];
44 const doubledNumbers = numbers.map((x) => x * 2);
45 console.log(doubledNumbers); // [2, 4, 6, 8, 10]
46 // Filter()
47 // The filter() method creates a new array with all elements that pass the test
48 // implemented by the provided function. It returns a new array with a length that
49 // is less than or equal to the original array, where each element is an element from
50 // the original array that meets the condition.
51 // Syntax
52 // arr.filter(callbackFunction(currentValue, index, array))
53 const numbers2 = [1, 2, 3, 4, 5];
54 const evenNumbers = numbers2.filter((x) => x % 2 === 0);
55 console.log(evenNumbers); // [2, 4]
56 // -----
57
58 // 3. What is the difference between 'let', 'const', and 'var'?
59 // var is the oldest way of declaring variables in JavaScript. It has a function
60 // scope, which means that a variable declared with var is accessible throughout
61 // the function it is declared in.
62 ✓ function example() {
63     var x = 10;
64     if (true) {
65         var x = 20; // same variable!
66         console.log(x); // outputs 20
67     }
68     console.log(x); // outputs 20
69 }
70 example();
71 // Let
72 // let is a newer way of declaring variables in JavaScript, introduced in ECMAScript
73 // 2015 (ES6). It has a block scope, which means that a variable declared with let
74 // is only accessible within the block it is declared in.
75 ✓ function example() {
76     let x = 10;
77     if (true) {
78         let x = 20; // different variable!
79         console.log(x); // outputs 20
80     }
81     console.log(x); // outputs 10
82 }
83 example();
84 // Const
85 // const is also a newer way of declaring variables in JavaScript, introduced in
86 // ECMAScript 2015 (ES6). It has the same block scope as let, but with an additional
87 // restriction: a variable declared with const cannot be reassigned.
88 ✓ function example() {
89     const obj = { foo: "bar" };

```

```

98     const obj = { foo: 'bar' };
99     obj.foo = "baz"; // works!
100    console.log(obj); // outputs { foo: 'baz' }
101  }
102  // Hoisting
103  // var variables are "hoisted" to the top of their scope, which means that they are
104  // moved to the top of the function or script, regardless of where they are actually
105  // declared. This can lead to unexpected behavior.
106  // let and const variables are not hoisted in the same way. They are only accessible after
107  // they are declared.
108  function example() {
109    console.log(x); // ReferenceError: x is not defined
110    let x = 10;
111  }
112  // -----
113  // 4. What is the difference between Pure and Impure functions?
114  // A pure function is a function that:
115  // Always returns the same output given the same inputs.
116  // Has no side effects, such as modifying external state or making API calls.
117  // Does not rely on external state or variables.
118  // In other words, a pure function is a self-contained, deterministic function that
119  // only depends on its input arguments and does not affect the outside world.
120  // Example of a pure function:
121  function add(a, b) {
122    return a + b;
123  }
124
125  // Impure Functions
126  // May return different outputs given the same inputs.
127  // Has side effects, such as modifying external state or making API calls.
128  // Relies on external state or variables.
129  // Impure functions can have unintended consequences, making them harder to
130  // reason about and test.
131  let counter = 0;
132
133  function increment() {
134    counter++;
135    return counter;
136  }
137  // }-----
138  // 5. What is the difference between for-in and for-of ?
139  // Answer:
140  // for-in Loop
141  // Purpose: The for-in loop is used to iterate over the enumerable properties of an object.
142  // Usage: It is primarily used with objects, but it can also be used with arrays (though
143  // not recommended for arrays).
144  const obj5 = { a: 1, b: 2, c: 3 };
145  for (let key in obj5) {
146    console.log(key, obj5[key]); // Outputs: a 1, b 2, c 3
147  }
148  // for-of Loop
149  // Purpose: The for-of loop is used to iterate over iterable objects, such as arrays, strings,
150  // Maps, Sets, etc.
151  // Usage: It is specifically designed for collections that are iterable.
152  const array = [1, 2, 3];
153  for (let value of array) {

```

```
142     console.log(value); // Outputs: 1, 2, 3
143 }
144 // 6. What are the differences between call(), apply() and bind() ?
145 // Answer
146 // call()
147 // Purpose: Invokes a function with a specified this value and individual arguments.
148 // Syntax: functionName.call(thisArg, arg1, arg2, ...);
149 function greet(greeting) {
150     console.log(greeting + ", " + this.name);
151 }
152
153 const person5 = { name: "Alice" };
154 greet.call(person5, "Hello"); // Outputs: Hello, Alice
155 //   apply()
156 // Purpose: Invokes a function with a specified this value and arguments provided as an
157 // Syntax: functionName.apply(thisArg, [argsArray]);
158 function greet(greeting) {
159     console.log(greeting + ", " + this.name);
160 }
161
162 const person = { name: "Bob" };
163 greet.apply(person, ["Hi"]); // Outputs: Hi, Bob
164
165 // bind()
166 // Purpose: Returns a new function that, when called, has its this keyword set to the pr
167 // sequence of arguments preceding any provided when the new function is called.
168 // Syntax: functionName.bind(thisArg, arg1, arg2, ...);
169 function greet(greeting) {
170     console.log(greeting + ", " + this.name);
171 }
172
173 const person9 = { name: "Charlie" };
174 const greetCharlie = greet.bind(person9);
175 greetCharlie("Hey"); // Outputs: Hey, Charlie
176
177 // Key Differences
178 // Invocation:
179
180 // call() and apply() immediately invoke the function.
181 // bind() returns a new function that can be invoked later.
182 // Argument Handling:
183
184 // call() takes arguments individually.
185 // apply() takes an array of arguments.
186 // bind() allows you to preset arguments that will be passed when the new function is ca
187 // Use Cases:
188
189 // Use call() when you want to invoke a function immediately with a specific this contex
190 // Use apply() when you want to invoke a function immediately with a specific this conte
191 // Use bind() when you want to create a new function with a specific this context that c
192
193 // 7. List out some key features of ES6 ?
```

```
194 // 8. what's the spread operator in javascript ?
195
196 // The spread operator in JavaScript, denoted by three consecutive dots (...), is a synt
197 // (like an array or an object) to be expanded in places where zero or more arguments c
198 // It is commonly used for array manipulation, function calls, and object cloning or me
199
200 // Key Uses of the Spread Operator:-
201 // Array Expansion
202
203 // You can use the spread operator to expand an array into individual elements.
204 const arr1 = [1, 2, 3];
205 const arr2 = [4, 5, ...arr1, 6]; // arr2 = [4, 5, 1, 2, 3, 6]
206
207 // Function Arguments
208 // It allows you to pass elements of an array as individual arguments to a function.
209
210 function sum(a, b, c) {
211   return a + b + c;
212 }
213
214 const numbers0 = [1, 2, 3];
215 console.log(sum(...numbers)); // Outputs: 6
216
217 // Object Expansion
218 // You can use the spread operator to create a new object by copying properties from an
219
220 const obj1 = { a: 1, b: 2 };
221 const obj2 = { ...obj1, c: 3 }; // obj2 = { a: 1, b: 2, c: 3 }
222
223 // Merging Arrays
224 // The spread operator can be used to merge multiple arrays into one.
225
226 const arr1 = [1, 2];
227 const arr2 = [3, 4];
228 const mergedArray = [...arr1, ...arr2]; // mergedArray = [1, 2, 3, 4]
229
230 // Cloning Arrays and Objects
231 // It provides a simple way to create shallow copies of arrays and objects.
232 const originalArray = [1, 2, 3];
233 const clonedArray = [...originalArray]; // clonedArray = [1, 2, 3]
234
235 const originalObject = { a: 1, b: 2 };
236 const clonedObject = { ...originalObject }; // clonedObject = { a: 1, b: 2 }
237
238 // Important Notes
239 // The spread operator creates a shallow copy of the array or object. This means that ne
240 // It is different from the rest operator, which is used to collect multiple elements in
241 // -----
242 // 9. What is rest operator in javascript ?
243 // The rest operator in JavaScript, denoted by three consecutive dots (...), is a syntax
244 // indefinite number of arguments as an array. It is commonly used in function parameter
245 // into a single array, making it easier to work with variable numbers of arguments.
246 // Key Uses of the Rest Operator
```

 main Last_Believe / jsinterview.js ↑ Top

CodeBlame

 Raw    

```
251     }
252
253     console.log(sum(1, 2, 3)); // Outputs: 6
254     console.log(sum(4, 5, 6, 7, 8)); // Outputs: 30
255
256     //   Destructuring Assignment
257     // The rest operator can be used in destructuring assignments to collect the remaining
258     // properties of an object into a new array or object.
259     const arr = [1, 2, 3, 4, 5];
260     const [first, second, ...rest] = arr; // first = 1, second = 2, rest = [3, 4, 5]
261
262     console.log(rest); // Outputs: [3, 4, 5]
263
264     //   Combining with Other Parameters
265     // You can combine the rest operator with other parameters in a function definition. How
266     // parameter must always be the last parameter.
267     function multiply(multiplier, ...numbers) {
268         return numbers.map((num) => num * multiplier);
269     }
270
271     console.log(multiply(2, 1, 2, 3)); // Outputs: [2, 4, 6]
272
273     // Important Notes
274     // The rest operator collects all remaining arguments into a single array, while the spr
275     // (also denoted by ...) expands an array into individual elements.
276     // You can only have one rest parameter in a function, and it must be the last parameter
277     // -----
278     // 10. What are DRY, KISS, YAGNI, SOLID Principles ?
279     // DRY, KISS, YAGNI, and SOLID are important software development principles that help c
280     // maintainable, efficient, and scalable code. Here's a brief overview of each principl
281
282     // 1. DRY (Don't Repeat Yourself)
283     // Definition: The DRY principle emphasizes the importance of reducing duplication in co
284     // of knowledge or logic should have a single, unambiguous representation within a syste
285     // Purpose: By avoiding code duplication, you reduce the risk of inconsistencies and mak
286     // easier to maintain. Changes need to be made in only one place, which minimizes the ch
287     // Example: Instead of writing the same function multiple times, create a single functio
288     // 2. KISS (Keep It Simple, Stupid)
289     // Definition: The KISS principle advocates for simplicity in design and implementation.
290     // as possible, avoiding unnecessary complexity.
291     // Purpose: Simple designs are easier to understand, maintain, and extend. Complexity ca
292     // Example: When designing a function, focus on the core functionality and avoid adding
293     // 3. YAGNI (You Aren't Gonna Need It)
294     // Definition: The YAGNI principle states that you should not add functionality until it
295     // add features based on assumptions about future needs, which can lead to bloated code.
296     // Purpose: By adhering to YAGNI, you keep the codebase lean and focused on current requ
297     // unnecessary features and minimizing maintenance overhead.
298     // Example: If a feature is not required for the current project scope, do not implement
```

```

299 // 4. SOLID Principles
300 // The SOLID principles are a set of five design principles aimed at making software des
301
302 // S: Single Responsibility Principle (SRP)
303
304 // A class should have only one reason to change, meaning it should have only one job or
305 // O: Open/Closed Principle (OCP)
306
307 // Software entities (classes, modules, functions, etc.) should be open for extension bu
308 // This means you should be able to add new functionality without changing existing code
309 // L: Liskov Substitution Principle (LSP)
310
311 // Objects of a superclass should be replaceable with objects of a subclass without affe
312 // This ensures that a subclass can stand in for its superclass.
313 // I: Interface Segregation Principle (ISP)
314
315 // Clients should not be forced to depend on interfaces they do not use. This principle
316 // specific interfaces rather than a large, general-purpose one.
317 // D: Dependency Inversion Principle (DIP)
318
319 // High-level modules should not depend on low-level modules; both should depend on abst
320 // should not depend on details; details should depend on abstractions. This promotes lo
321 // Summary
322 // DRY: Avoid code duplication.
323 // KISS: Keep designs simple and straightforward.
324 // YAGNI: Do not implement features until they are necessary.
325 // SOLID: A set of five principles (SRP, OCP, LSP, ISP, DIP) that guide software design
326 // -----
327 // 11. What is temporal dead zone ?
328 // The Temporal Dead Zone (TDZ) is a concept in JavaScript that refers to the period of
329
330 // Key Points about Temporal Dead Zone
331 // Scope: The TDZ applies to block-scoped variables declared with let and const. It exis
332 // Initialization: A variable declared with let or const is not hoisted in the same way
333 // hoisted and initialized to undefined, let and const declarations are hoisted but rema
334 // ReferenceError: If you try to access a variable in the TDZ, JavaScript will throw a R
335 // Example of Temporal Dead Zone
336 // Here's an example to illustrate the concept:
337 ✓ function example() {
338     console.log(x); // ReferenceError: Cannot access 'x' before initialization
339     let x = 10; // x is in the TDZ until this line is executed
340     console.log(x); // Outputs: 10
341 }
342 example();
343 // In this example:
344 // The first console.log(x) tries to access x before it has been initialized, resulting
345 // The second console.log(x) is executed after x has been initialized, so it outputs 10.
346 // Summary
347 // The Temporal Dead Zone is an important concept to understand when working with let ar
348 // It helps prevent the use of variables before they are initialized, promoting better
349 // reducing potential bugs. By being aware of the TDZ, developers can write cleaner and
350 // -----
351 // 12. Different ways to create object in javascript ?

```

```
351 // 12. Different ways to create object in javascript .
352 // 1. Object Literal
353 // The simplest way to create an object is by using an object literal. This method allow
354 // object with properties and methods directly
355 const person55 = {
356   name: "Alice",
357   age: 30,
358   greet: function () {
359     console.log(`Hello, my name is ${this.name}`);
360   },
361 };
362 person55.greet(); // Outputs: Hello, my name is Alice
363
364 // 2. Constructor Function
365 // You can create objects using constructor functions. A constructor function is a regul
366 // 3. ES6 Classes
367 // With the introduction of ES6, you can create objects using class syntax, which is a m
368 ✓ class Person {
369   constructor(name, age) {
370     this.name = name;
371     this.age = age;
372   }
373
374   greet() {
375     console.log(`Hello, my name is ${this.name}`);
376   }
377 }
378
379 const person2 = new Person("Charlie", 28);
380 person2.greet(); // Outputs: Hello, my name is Charlie
381 // 4. Object.create()
382 // The Object.create() method creates a new object with the specified prototype object a
383 // for creating objects that inherit from other objects.
384 const animal = {
385   speak: function () {
386     console.log("Animal speaks");
387   },
388 };
389
390 const dog = Object.create(animal);
391 dog.bark = function () {
392   console.log("Dog barks");
393 };
394
395 dog.speak(); // Outputs: Animal speaks
396 dog.bark(); // Outputs: Dog barks
397 // 5. Factory Function
398 // A factory function is a function that returns a new object each time it is called. Th
399 // multiple instances of an object without using the new keyword.
400 ✓ function createPerson(name, age) {
401   return {
402     name: name,
403     age: age,
```

```

404     greet: function () {
405         console.log(`Hello, my name is ${this.name}`);
406     },
407 };
408 }
409
410 const person3 = createPerson("David", 22);
411 person3.greet(); // Outputs: Hello, my name is David
412 // 6. Using new Object()
413 // You can also create an object using the Object constructor, although this method is 1
414 const person4 = new Object();
415 person4.name = "Eve";
416 person4.age = 35;
417 person4.greet = function () {
418     console.log(`Hello, my name is ${this.name}`);
419 };
420 person4.greet(); // Outputs: Hello, my name is Eve
421 // Summary
422 // Object Literal: Simple and straightforward way to create objects.
423 // Constructor Function: Allows for creating multiple instances of an object.
424 // ES6 Classes: Modern syntax for creating objects and handling inheritance.
425 // Object.create(): Creates an object with a specified prototype.
426 // Factory Function: Returns a new object each time it is called.
427 // Using new Object(): Less common method for creating objects
428 // -----
429 // 13. Whats the difference between Object.keys, values and entries
430 // 1. Object.keys(obj)
431 // Returns: array of property names (keys)
432 // Type: string[]
433 const user = { name: "Sai", age: 24, country: "India" };
434
435 console.log(Object.keys(user));
436 // ["name", "age", "country"]
437
438 // 2. Object.values(obj)
439 // Returns: array of property values
440 // Type: any[] (depends on the values stored)
441
442 // ✅ Example:
443 console.log(Object.values(user));
444 // ["Sai", 24, "India"]
445
446
447 // 3. Object.entries(obj)
448 // Returns: array of key-value pairs
449 // Type: [string, any][] (array of arrays)
450 // ✅ Example:
451 console.log(Object.entries(user));
452 // [["name", "Sai"], ["age", 24], ["country", "India"]]
453
454 // | Method | Returns | Example Result for `{a:1, b:2, c:3}`
455 // | ----- | ----- | -----

```

```

456 // | Object.keys(obj) | Array of keys (strings) | ["a", "b"]
457 // | `Object.values(obj)` | Array of values | `[1, 2]`
458 // | `Object.entries(obj)` | Array of `[key, value]` pairs | `[["a",1], ["b",2]]`
459
460 // -----
461 // 14. Whats the difference between Object.freeze() vs Object.seal()
462 // Object.freeze(obj)
463
464 // Purpose: Makes an object completely immutable.
465 // What happens:
466 // ✗ Cannot add new properties
467 // ✗ Cannot remove properties
468 // ✗ Cannot change values of existing properties
469 // ✗ Cannot reconfigure properties (writable/configurable)
470 // ✓ Example:
471
472 const user = { name: "Sai", age: 24 };
473 Object.freeze(user);
474
475 user.age = 25; // ✗ ignored
476 user.city = "India"; // ✗ ignored
477 delete user.name; // ✗ ignored
478
479 console.log(user); // { name: "Sai", age: 24 }
480 // ♦ Object.seal(obj)
481 // Purpose: Seals an object, meaning you can't add or remove properties, but you can sti
482 // What happens:
483 // ✗ Cannot add new properties
484 // ✗ Cannot remove properties
485 // ✓ Can change values of existing properties
486 // ✗ Cannot reconfigure properties (like making them non-writable)
487 // ✓ Example:
488
489 const user = { name: "Sai", age: 24 };
490 Object.seal(user);
491
492 user.age = 25; // ✓ works
493 user.city = "India"; // ✗ ignored
494 delete user.name; // ✗ ignored
495
496 console.log(user); // { name: "Sai", age: 25 }
497
498 // | Feature | `Object.freeze()` | `Object.seal()` |
499 // | ----- | ----- | ----- |
500 // | Add new properties | ✗ No | ✗ No |
501 // | Delete properties | ✗ No | ✗ No |
502 // | Modify existing values | ✗ No | ✓ Yes |
503 // | Reconfigure property attrs | ✗ No | ✗ No |
504 // Use Object.freeze() when you want a fully immutable object.
505
506 // Use Object.seal() when you want to lock the shape of the object but still allow updat
507 // -----
508 // 15. What is a polyfill in javascript ?

```

```

509 // Example 1 - Polyfill for Array.prototype.includes()
510 // Some old browsers (like Internet Explorer) don't support includes().
511 // We can polyfill it ourselves:
512
513 if (!Array.prototype.includes) {
514     Array.prototype.includes = function (searchElement, fromIndex) {
515         return this.indexOf(searchElement, fromIndex) !== -1;
516     };
517 }
518
519 console.log([1, 2, 3].includes(2)); // true
520
521
522 // Here, if the browser doesn't have includes(), we define it using older indexOf().
523
524 // ✅ Example 2 - Polyfill for Object.assign()
525 if (typeof Object.assign !== "function") {
526     ✓ Object.assign = function (target, ...sources) {
527         sources.forEach(src => {
528             for (let key in src) {
529                 if (Object.prototype.hasOwnProperty.call(src, key)) {
530                     target[key] = src[key];
531                 }
532             }
533         });
534         return target;
535     };
536 }
537
538 // const obj1 = { a: 1 };
539 // const obj2 = { b: 2 };
540 // console.log(Object.assign({}, obj1, obj2)); // { a:1, b:2 }
541
542 // ♦ Difference Between Polyfill and Transpiler
543
544 // Polyfill → Runtime code that adds missing methods/features.
545
546 // Transpiler (like Babel) → Converts modern JS syntax (e.g. ES6+) into older syntax (e.
547
548 // 👉 Example:
549
550 // Babel converts () => {} into a normal function.
551
552 // A polyfill provides missing features like Promise, fetch, Array.includes, etc.
553
554 // 🌟 In simple words:
555 // A polyfill is a patch for browsers that don't know how to do something new in JavaScript
556 // -----
557 // -----
558 // -----
559
560 // 16. What is a generator function in JavaScript?

```

```
561
562 // A generator function is a special function that can pause its execution and later res
563
564 // Syntax: function* myGenerator() { yield 1; yield 2; }
565
566 // You use yield to pause execution and next() to resume.
567
568 // Example:
569
570 // function* numbers() {
571 //   yield 1;
572 //   yield 2;
573 //   yield 3;
574 // }
575 // const gen = numbers();
576 // console.log(gen.next()); // {value: 1, done: false}
577 // console.log(gen.next()); // {value: 2, done: false}
578 // console.log(gen.next()); // {value: 3, done: false}
579 // console.log(gen.next()); // {value: undefined, done: true}
580
581
582 // 17. What is prototype in JavaScript?
583
584 // Every JavaScript object has a prototype property, which is used to share methods and
585
586 // It allows inheritance in JS.
587
588 // Example:
589
590 // function Person(name) {
591 //   this.name = name;
592 // }
593 // Person.prototype.greet = function() {
594 //   console.log(`Hello, ${this.name}`);
595 // };
596 // const person1 = new Person("Saikrishna");
597 // person1.greet(); // Hello, Saikrishna
598
599
600 // 18. What is IIFE? (Immediately Invoked Function Expression)
601
602 // A function that runs immediately after it's defined.
603
604 // Helps avoid polluting the global scope.
605
606 // Syntax:
607
608 // (function() {
609 //   console.log("IIFE executed!");
610 // })();
611
612
613 // 19. What is CORS? (Cross-Origin Resource Sharing)
```

```
613 // 19. What is CORS? (Cross-Origin Resource Sharing)
614
615 // CORS is a browser security feature that restricts web pages from making requests to a
616
617 // Servers can allow cross-origin requests by sending specific headers like Access-Control-Allow-Origin
618
619 // 20. What are the different datatypes in JavaScript?
620
621 // Primitive: String, Number, Boolean, Null, Undefined, Symbol, BigInt
622
623 // Non-primitive (Reference): Object, Array, Function
624
625 // 21. Difference between TypeScript and JavaScript:
626
627 // Feature   JavaScript   TypeScript
628 // Typing     Dynamic   Static (optional)
629 // Compilation   Interpreted   Compiled to JS
630 // Tooling     Basic       Advanced (autocompletion, type-checking)
631 // ES6+ Features   Supported   Fully supported with extra types
632
633 // 22. Authentication vs Authorization:
634
635 // Authentication: Verifying who a user is (login).
636
637 // Authorization: Verifying what a user can access (permissions).
638
639 // 23. Difference between null and undefined:
640
641 // Feature   null       undefined
642 // Value     Explicitly assigned empty value   Default value for uninitialized variables
643 // Type      object     undefined
644
645 // 24. What is the output of 3 + 2 + "7"?
646
647 // Output: "57"
648
649 // Explanation: JS evaluates left-to-right: (3+2)=5, then 5 + "7" → string concatenation
650
651 // 25. Slice vs Splice:
652
653 // Feature   slice     splice
654 // Modifies original array   No   Yes
655 // Returns   New array   Removed elements
656 // Syntax    arr.slice(start, end)   arr.splice(start, deleteCount, items...)
657
658 // 26. What is destructuring?
659
660 // A way to extract values from arrays or objects into variables.
661
662 // Example:
663
664 // const [a, b] = [1, 2]; // array destructuring
665 // const {name, age} = {name: "Sai", age: 25}; // object destructuring
```

```
666
667
668 // 27. What is setTimeout in JavaScript?
669
670 // Executes a function once after a specified delay (in milliseconds).
671
672 // Example:
673
674 // setTimeout(() => {
675 //   console.log("Executed after 2 seconds");
676 // }, 2000);
677
678
679 // 28. What is setInterval in JavaScript?
680
681 // Executes a function repeatedly at a specified interval (in milliseconds).
682
683 // Example:
684
685 // setInterval(() => {
686 //   console.log("Repeating every 1 second");
687 // }, 1000);
688
689
690 // 29. What are Promises in JavaScript?
691
692 // A Promise represents a value that may be available now, later, or never.
693
694 // Used to handle asynchronous operations.
695
696 // Example:
697
698 // const promise = new Promise((resolve, reject) => {
699 //   setTimeout(() => resolve("Done!"), 1000);
700 // });
701 // promise.then(result => console.log(result)); // "Done!" after 1 sec
702
703
704 // 30. What is a call stack in JavaScript?
705
706 // A call stack is a data structure that keeps track of function calls in execution order.
707
708 // JS is single-threaded, so it uses the call stack to manage execution.
709
710 // Example:
711
712 // function a() { b(); }
713 // function b() { c(); }
714 // function c() { console.log("Hello"); }
715 // a(); // Call stack: a → b → c
716
717
```

```
// if you want, i can also make a one-page "cheat sheet" with all these JS concepts and
719
720 // Do you want me to do that?
721
722 // You said:
723 // // 31. What is a closure ?
724 // // 32. What are callbacks in javascript ?
725 // // 33. What are Higher Order Functions in javascript ?
726 // // 34. What is the difference between == and === in javascript ?
727 // // 35. Is javascript a dynamically typed language or a statically typed language
728 // // 36. What is the difference between Indexeddb and sessionStorage ?
729 // // 37. What are Interceptors ?
730 // // 38. What is Hoisting ?
731 // // 39. What are the differences let, var and const ?
732 // // 41. Differences between Promise.all, allSettled, any, race ?
733 // // 42. What are limitations of arrow functions?
734 // ChatGPT said:
735
736 // Here's a detailed explanation for your next set of JavaScript questions:
737
738 // 31. What is a closure?
739
740 // A closure is a function that remembers and can access variables from its outer scope,
741
742 // Example:
743
744 // function outer() {
745 //   let count = 0;
746 //   return function inner() {
747 //     count++;
748 //     console.log(count);
749 //   };
750 // }
751 // const counter = outer();
752 // counter(); // 1
753 // counter(); // 2
754
755
756 // Here, inner forms a closure over count.
757
758 // 32. What are callbacks in JavaScript?
759
760 // A callback is a function passed as an argument to another function, executed after so
761
762 // Example:
763
764 // function greet(name, callback) {
765 //   console.log("Hello " + name);
766 //   callback();
767 // }
768 // greet("Sai", () => console.log("Callback executed!"));
769
770
```

```
771 // 33. What are Higher Order Functions (HOF) in JavaScript?
772
773 // A function that takes another function as an argument or returns a function.
774
775 // Examples: map, filter, reduce.
776
777 // const numbers = [1, 2, 3];
778 // const doubled = numbers.map(n => n * 2); // map is HOF
779 // console.log(doubled); // [2, 4, 6]
780
781
782 // 34. Difference between == and ===:
783
784 // Operator Comparison Type Conversion
785 // == Checks value equality Performs type coercion
786 // === Checks value & type equality No type coercion
787 // 5 == "5"; // true
788 // 5 === "5"; // false
789
790
791 // 35. Is JavaScript dynamically typed or statically typed?
792
793 // JavaScript is dynamically typed: variable types are determined at runtime and can cha
794
795 // let x = 10; // number
796 // x = "Hello"; // now string
797
798
799 // 36. Difference between IndexedDB and SessionStorage:
800
801 // Feature IndexedDB SessionStorage
802 // Type NoSQL DB in browser Key-value storage
803 // Capacity Large (~50MB+) Small (~5MB)
804 // Persistence Persistent until explicitly deleted Cleared on tab close
805 // Structure Stores objects, supports transactions Stores strings only
806
807 // 37. What are Interceptors?
808
809 // In frameworks like Axios, interceptors are functions that intercept requests or respo
810
811 // Used for: adding headers, logging, handling errors globally.
812
813 // axios.interceptors.request.use(config => {
814 //   config.headers['Authorization'] = 'Bearer token';
815 //   return config;
816 // });
817
818
819 // 38. What is Hoisting?
820
821 // JS moves variable and function declarations to the top of their scope during compilat
822
```

```

823 // Example:
824
825 // console.log(a); // undefined
826 // var a = 5;      // declaration is hoisted
827
828
829 // Note: let and const are hoisted but cannot be accessed before declaration (Temporal D
830
831 // 39. Differences between let, var, and const:
832
833 // Feature  var let const
834 // Scope    Function    Block    Block
835 // Hoisting Yes, initialized undefined Yes, but TDZ    Yes, but TDZ
836 // Reassignable Yes Yes No
837 // Redeclarable Yes No No
838
839 // 41. Differences between Promise.all, allSettled, any, race:
840
841 // Method    Behavior
842 // Promise.all([p1,p2]) Resolves when all promises resolve; rejects if any fail
843 // Promise.allSettled([p1,p2]) Resolves when all promises settle (resolve or reject) wi
844 // Promise.any([p1,p2]) Resolves when any promise resolves; rejects only if all fail
845 // Promise.race([p1,p2]) Resolves or rejects as soon as any promise resolves/rejects
846
847 // 42. Limitations of arrow functions:
848
849 // No this binding – uses this from the enclosing context.
850
851 // Cannot be used as constructors (new fails).
852
853 // No arguments object (use rest parameters ...args).
854
855 // Cannot use yield – cannot be generator functions.
856
857 // I can also combine all 30+ JS interview questions with examples into a single, easy-t
858
859 // Do you want me to create that?
860
861 // You said:
862 // // 43. What is difference between find vs findIndex ?
863 // // 44. What is tree shaking in javascript ?
864 // // 45. What is the main difference between Local Storage and Session storage ?
865 // // 46. What is eval() ?
866 // // 47. What is the difference between Shallow copy and deep copy ?
867 // // 48. What are the difference between undeclared and undefined variables ?
868 // // 49. What is event bubbling
869 // // 50. What is event capturing ?
870 // // 51. What are cookies ?
871 // // 52. typeof operator ?
872 // // 53. What is this in javascript and How it behaves in various scenarios ?
873 // // 54. How do you optimize the performance of application
874 // // 55. What is meant by debouncing and throttling
875 // ChatGPT said:

```

```
875 // console.log();
876
877 // Here's a detailed explanation for your next set of JavaScript questions:
878
879 // 43. Difference between find and findIndex:
880
881 // Method    Returns    Description
882 // find()    The first element that satisfies the condition    Returns the element itself
883 // findIndex()    The index of the first element that satisfies the condition    Returns -1 if not found
884 // const arr = [10, 20, 30];
885 // arr.find(x => x > 15);    // 20
886 // arr.findIndex(x => x > 15); // 1
887
888
889 // 44. What is tree shaking in JavaScript?
890
891 // Tree shaking is a technique to remove unused code during the bundling process (common in ES6 modules)
892
893 // Helps reduce bundle size for production.
894
895 // 45. Main difference between Local Storage and Session Storage:
896
897 // Feature    LocalStorage    SessionStorage
898 // Lifetime    Persistent until cleared    Cleared when tab closes
899 // Storage limit    ~5MB    ~5MB
900 // Scope    Shared across tabs of same origin    Specific to tab/window
901
902 // 46. What is eval()?
903
904 // eval() executes a string as JavaScript code.
905
906 // Example:
907
908 // eval("console.log(2 + 3)"); // 5
909
910
911 // ⚠️ Security risk: Can execute malicious code; avoid using it.
912
913 // 47. Difference between Shallow copy and Deep copy:
914
915 // Feature    Shallow Copy    Deep Copy
916 // Nested objects    References copied    Values copied recursively
917 // Libraries/Methods    Object.assign, spread ({...obj})    structuredClone, JSON.parse(JSON.stringify(obj))
918 // let obj = { a: 1, b: { c: 2 } };
919 // let shallow = {...obj}; // nested object is reference
920 // let deep = JSON.parse(JSON.stringify(obj)); // fully copied
921
922
923 // 48. Difference between undeclared and undefined variables:
924
925 // Type    Description
926 // Undeclared    Not declared at all in any scope; accessing throws ReferenceError
927 // Undefined    Declared but not assigned a value; default value is undefined
```

```
928 // console.log(x); // ReferenceError (undeclared)
929 // let y;
930 // console.log(y); // undefined
931
932
933 // 49. What is event bubbling?
934
935 // Event propagates from child element to parent elements in the DOM.
936
937 // Default behavior in JavaScript.
938
939 // <div onclick="console.log('parent')">
940 //   <button onclick="console.log('child')">Click</button>
941 // </div>
942 // // Click → logs: "child" then "parent"
943
944
945 // 50. What is event capturing?
946
947 // Event propagates from parent element to child elements (opposite of bubbling).
948
949 // Use addEventListener with capture = true.
950
951 // element.addEventListener('click', handler, true); // capturing phase
952
953
954 // 51. What are cookies?
955
956 // Small pieces of data stored in the browser and sent to the server with requests.
957
958 // Used for session management, personalization, tracking.
959
960 // 52. typeof operator:
961
962 // Returns a string indicating the type of a variable.
963
964 // typeof 5; // "number"
965 // typeof "hello"; // "string"
966 // typeof {}; // "object"
967 // typeof undefined; // "undefined"
968 // typeof null; // "object" (historical bug)
969
970
971 // 53. What is this in JavaScript and how it behaves:
972
973 // Scenario Value of this
974 // Global context window (browser) or global (Node.js)
975 // Function call window (non-strict) or undefined (strict)
976 // Method call Object that owns the method
977 // Constructor (new) New instance created
978 // Arrow function this from enclosing scope (lexical)
979
```

```
980 // 54. How do you optimize application performance?
981
982 // Minimize DOM manipulations.
983
984 // Use lazy loading & code splitting.
985
986 // Debounce/throttle expensive events.
987
988 // Use requestAnimationFrame for animations.
989
990 // Optimize images & assets.
991
992 // Avoid memory leaks.
993
994 // Use efficient algorithms & data structures.
995
996 // 55. What is meant by debouncing and throttling?
997
998 // Debouncing: Ensures a function runs only after a delay since the last call.
999
1000 // Throttling: Ensures a function runs at most once every interval, no matter how many t
1001
1002 // // Debounce
1003 // function debounce(func, delay) {
1004 //   let timer;
1005 //   return function(...args) {
1006 //     clearTimeout(timer);
1007 //     timer = setTimeout(() => func.apply(this, args), delay);
1008 //   };
1009 // }
1010
1011 // // Throttle
1012 // function throttle(func, limit) {
1013 //   let lastCall = 0;
1014 //   return function(...args) {
1015 //     const now = Date.now();
1016 //     if (now - lastCall >= limit) {
1017 //       lastCall = now;
1018 //       func.apply(this, args);
1019 //     }
1020 //   };
1021 // }
1022
1023
1024 // If you want, I can now combine all 55+ JavaScript interview Q&A into a single, neatly
1025
1026 // Do you want me to create that?
1027
1028 // You said:
1029
1030 // // 1. What is React?
1031 // // 2. What is useMemo ?
1032 // // 3. What are the features of React ?
```

```
1033 // // 4. What is JSX ?
1034 // // 5. What is DOM ?
1035 // // 6. What is Virtual Dom ?
1036 // // 7. What is component life cycle of React class component ?
1037 // // 8. What are fragments in react ?
1038 // // 9. What are props in react ?
1039 // // 10. What are synthetic events in react ?
1040 // // 11. What are the difference between Package.json and Package.lock.json ?
1041 // // 12. What are the differences between client side and server side rendering ?
1042 // // 13. What is state in Reactjs?
1043 // // 14. What are props ?
1044 // // 15. What are the differences between State and Props in react ?
1045 // ChatGPT said:
1046
1047 // Here's a detailed explanation for your React questions:
1048
1049 // 1. What is React?
1050
1051 // React is a JavaScript library for building user interfaces, mainly for single-page ap
1052
1053 // Maintained by Facebook, React allows component-based architecture and efficient UI up
1054
1055 // 2. What is useMemo?
1056
1057 // useMemo is a React hook that memoizes a value to avoid expensive recalculations on ev
1058
1059 // Syntax: const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b]);
1060
1061 // Useful for performance optimization.
1062
1063 // 3. Features of React:
1064
1065 // JSX (JavaScript + HTML) for templating.
1066
1067 // Component-based architecture.
1068
1069 // Virtual DOM for efficient updates.
1070
1071 // One-way data binding (parent → child).
1072
1073 // Hooks for managing state and lifecycle in functional components.
1074
1075 // Declarative UI.
1076
1077 // 4. What is JSX?
1078
1079 // JSX (JavaScript XML) is a syntax extension for JavaScript that looks like HTML.
1080
1081 // Example:
1082
1083 // const element = <h1>Hello, React!</h1>;
1084
```

```
1085
1086 // JSX is transpiled to React.createElement() calls.
1087
1088 // 5. What is DOM?
1089
1090 // DOM (Document Object Model) is a tree-like representation of HTML elements in the bro
1091
1092 // 6. What is Virtual DOM?
1093
1094 // Virtual DOM is a lightweight copy of the real DOM in memory.
1095
1096 // React diffs the virtual DOM and real DOM to update only changed elements, improving p
1097
1098 // 7. Component lifecycle of React class component:
1099
1100 // Mounting: constructor(), static getDerivedStateFromProps(), render(), componentDidMou
1101
1102 // Updating: static getDerivedStateFromProps(), shouldComponentUpdate(), render(), getSt
1103
1104 // Unmounting: componentWillUnmount()
1105
1106 // Error handling: componentDidCatch()
1107
1108 // 8. What are Fragments in React?
1109
1110 // Fragments allow grouping multiple elements without adding extra nodes to the DOM.
1111
1112 // <>
1113 //   <h1>Title</h1>
1114 //   <p>Description</p>
1115 // </>
1116
1117
1118 // 9. What are props in React?
1119
1120 // Props (properties) are read-only inputs passed from parent to child components.
1121
1122 // function Child({ name }) {
1123 //   return <h1>Hello {name}</h1>;
1124 // }
1125 // <Child name="Saikrishna" />
1126
1127
1128 // 10. What are Synthetic Events in React?
1129
1130 // Synthetic events are cross-browser wrappers around native browser events.
1131
1132 // They behave consistently across different browsers.
1133
1134 // <button onClick={handleClick}>Click</button>
1135
1136
1137 // 11. Difference between package.json and package-lock.json:
```

```

1137 // 11. Difference between package.json and package-lock.json.
1138
1139 // Feature   package.json   package-lock.json
1140 // Purpose   Lists dependencies & scripts   Records exact dependency versions
1141 // Created   Manually or npm init   Automatically on npm install
1142 // Updates   Can change version ranges   Locks versions to ensure consistent installs
1143
1144 // 12. Differences between Client-side Rendering (CSR) and Server-side Rendering (SSR):
1145
1146 // Feature   CSR   SSR
1147 // Rendering   Browser   Server
1148 // First load   Slower   Faster
1149 // SEO   Poor   Good
1150 // Example   React SPA   Next.js
1151
1152 // 13. What is state in React?
1153
1154 // State is a local, mutable data that controls component behavior and re-renders when u
1155
1156 // const [count, setCount] = useState(0);
1157
1158
1159 // 14. What are props?
1160
1161 // Props are immutable inputs passed from parent to child to configure or display data i
1162
1163 // 15. Differences between State and Props in React:
1164
1165 // Feature   State   Props
1166 // Mutability   Mutable   Immutable
1167 // Managed by   Component itself   Parent component
1168 // Purpose   Store dynamic data   Pass data/configuration
1169 // Triggers render   Yes   Yes (when parent changes)
1170
1171 // If you want, I can combine all your React + JavaScript interview questions into one u
1172
1173 // Do you want me to do that?
1174
1175 // You said:
1176 // // 16. What is props drilling ?
1177 // // 17. What are the disadvantages of props drilling and How we can avoid props drill
1178 // // 18. What are Pure components in React ?
1179 // // 19. What are Ref's in React?
1180 // // 20. What is meant by forward ref ?
1181 // // 21. What are Error boundaries ?
1182 // // 22. What are Higher order components in react ?
1183 // // 23. What are the differences between controlled and uncontrolled components ?
1184 // // 24. What is useCallback ?
1185 // // 25. What are the differences between useMemo and useCallback ?
1186 // // 26. What are keys in React ?
1187 // // 27. What is Lazy loading in React ?
1188 // // 28. What is suspense in React ?
1189 // // 29. What are custom hooks ?

```

```
1190 // // 30. What is useReducer hook ?
1191 // ChatGPT said:
1192
1193 // Here's a detailed explanation for your next set of React questions:
1194
1195 // 16. What is props drilling?
1196
1197 // Props drilling occurs when props are passed through multiple intermediate components
1198
1199 // Example: Grandparent → Parent → Child → DeepChild
1200
1201 // 17. Disadvantages of props drilling & how to avoid it:
1202
1203 // Disadvantages:
1204
1205 // Makes components tightly coupled.
1206
1207 // Hard to maintain in large apps.
1208
1209 // Solutions:
1210
1211 // Context API
1212
1213 // State management libraries like Redux, MobX, Zustand
1214
1215 // 18. What are Pure Components in React?
1216
1217 // Pure Components only re-render if their props or state change (shallow comparison).
1218
1219 // Class-based: extends React.PureComponent
1220
1221 // Helps optimize performance by preventing unnecessary renders.
1222
1223 // 19. What are Refs in React?
1224
1225 // Refs provide a way to access DOM nodes or React elements directly.
1226
1227 // const inputRef = useRef();
1228 // <input ref={inputRef} />;
1229 // inputRef.current.focus();
1230
1231
1232 // 20. What is meant by forwardRef?
1233
1234 // forwardRef allows a parent component to pass a ref to a child component.
1235
1236 // Example:
1237
1238 // const Child = React.forwardRef((props, ref) => <input ref={ref} />);
1239 // const ref = React.createRef();
1240 // <Child ref={ref} />;
1241
```

```
1242
1243 // 21. What are Error Boundaries?
1244
1245 // Components that catch JavaScript errors in their child components, log them, and disp
1246
1247 // Only works for class components.
1248
1249 // class ErrorBoundary extends React.Component {
1250 //   state = { hasError: false };
1251 //   static getDerivedStateFromError() { return { hasError: true }; }
1252 //   componentDidCatch(error, info) { console.log(error, info); }
1253 //   render() { return this.state.hasError ? <h1>Something went wrong</h1> : this.props.
1254 // }
1255
1256
1257 // 22. What are Higher Order Components (HOC) in React?
1258
1259 // HOC is a function that takes a component and returns a new component with enhanced fu
1260
1261 // Example: withAuth(Component)
1262
1263 // 23. Differences between Controlled and Uncontrolled Components:
1264
1265 // Feature    Controlled    Uncontrolled
1266 // Data handling    React state    DOM state
1267 // Update    via onChange    via ref
1268 // Example    <input value={state} onChange={handleChange} /> <input ref={inputRef} />
1269
1270 // 24. What is useCallback?
1271
1272 // useCallback memoizes a function, so it doesn't get recreated on every render.
1273
1274 // const memoizedFn = useCallback(() => { doSomething(); }, [dependencies]);
1275
1276
1277 // Useful when passing functions to child components to prevent unnecessary re-renders.
1278
1279 // 25. Differences between useMemo and useCallback:
1280
1281 // Feature    useMemo    useCallback
1282 // Returns    Memoized value    Memoized function
1283 // Usage    Expensive calculations    Functions passed as props
1284
1285 // 26. What are keys in React?
1286
1287 // Keys help React identify which items in a list have changed, added, or removed.
1288
1289 // {items.map(item => <li key={item.id}>{item.name}</li>)}
1290
1291
1292 // 27. What is Lazy Loading in React?
1293
1294 // Lazy loading loads components only when needed instead of upfront.
```

```
1295
1296 // const LazyComponent = React.lazy(() => import('./LazyComponent'));
1297
1298
1299 // 28. What is Suspense in React?
1300
1301 // Suspense wraps lazy-loaded components to show a fallback UI while the component is lo
1302
1303 // <Suspense fallback={<div>Loading...</div>}>
1304 //   <LazyComponent />
1305 // </Suspense>
1306
1307
1308 // 29. What are Custom Hooks?
1309
1310 // Functions that reuse stateful logic across components.
1311
1312 // Naming convention: useSomething.
1313
1314 // function useCounter() {
1315 //   const [count, setCount] = useState(0);
1316 //   const increment = () => setCount(c => c + 1);
1317 //   return { count, increment };
1318 // }
1319
1320
1321 // 30. What is useReducer hook?
1322
1323 // useReducer is used for complex state logic with multiple sub-values.
1324
1325 // Similar to Redux reducer.
1326
1327 // const initialState = { count: 0 };
1328 // function reducer(state, action) {
1329 //   switch(action.type) {
1330 //     case 'increment': return { count: state.count + 1 };
1331 //     default: return state;
1332 //   }
1333 // }
1334 // const [state, dispatch] = useReducer(reducer, initialState);
1335 // dispatch({ type: 'increment' });
1336
1337
1338 // If you want, I can combine all your 1-55 JavaScript & React questions into one comple
1339
1340 // Do you want me to do that?
1341
1342 // You said:
1343 // // 1. What is the difference between id and class attributes?
1344 // // 2. How does the doctype declaration affect a webpage?
1345 // // 3. Explain semantic HTML and its importance.
1346 // // 4. What are meta tags, and why are they used?
```

```
1347 // // 5. What is the difference between span and div tags?
1348 // // 6. How does the defer attribute in a script tag work?
1349 // // 7. What is the difference between inline, block, and inline-block elements?
1350 // // 8. How do you optimize images for web performance in HTML?
1351 // // 9. What are custom data attributes, and how are they used?
1352 // // 10. How would you create an accessible custom tooltip using only HTML?
1353 // ChatGPT said:
1354
1355 // Here's a detailed explanation for your HTML questions:
1356
1357 // 1. Difference between id and class attributes:
1358
1359 // Attribute    Description    Uniqueness    Usage
1360 // id           Identifies a single element Must be unique on a page    Targeting specific elements
1361 // class        Identifies one or more elements Can be shared across elements    Applying styles
1362 // <div id="header"></div>
1363 // <div class="card"></div>
1364 // <div class="card"></div>
1365
1366
1367 // 2. How does the <!DOCTYPE> declaration affect a webpage?
1368
1369 // Declares the HTML version and rendering mode for the browser.
1370
1371 // Modern HTML uses: <!DOCTYPE html> → triggers standards mode instead of quirks mode.
1372
1373 // Helps ensure consistent rendering across browsers.
1374
1375 // 3. Explain semantic HTML and its importance:
1376
1377 // Semantic HTML uses meaningful tags to describe content.
1378
1379 // Examples: <header>, <nav>, <article>, <footer>.
1380
1381 // Importance:
1382
1383 // Improves accessibility (screen readers).
1384
1385 // Helps SEO.
1386
1387 // Makes code readable & maintainable.
1388
1389 // 4. What are meta tags, and why are they used?
1390
1391 // <meta> tags provide metadata about the webpage, like charset, viewport, or description.
1392
1393 // Example:
1394
1395 // <meta charset="UTF-8">
1396 // <meta name="viewport" content="width=device-width, initial-scale=1.0">
1397 // <meta name="description" content="A sample webpage">
1398
1399
```

```
1400 // Used for SEO, responsiveness, and page info.
1401
1402 // 5. Difference between span and div tags:
1403
1404 // Tag   Type   Purpose
1405 // div   Block-level Groups content into sections
1406 // span  Inline  Groups small pieces of content inside other elements
1407 // <div>This is a block</div>
1408 // <span>This is inline</span>
1409
1410
1411 // 6. How does the defer attribute in a script tag work?
1412
1413 // <script src="script.js" defer></script> loads the script asynchronously but executes
1414
1415 // Ensures DOM is ready before script runs.
1416
1417 // 7. Difference between inline, block, and inline-block elements:
1418
1419 // Type Behavior   Examples
1420 // Inline   Doesn't start a new line, width & height ignored   <span>, <a>
1421 // Block    Starts on new line, full-width by default         <div>, <p>
1422 // Inline-block Like inline but respects width & height <img>, <button>
1423
1424 // 8. How do you optimize images for web performance in HTML?
1425
1426 // Use compressed formats (WebP, JPEG, PNG).
1427
1428 // Use responsive images (srcset).
1429
1430 // Lazy load images: <img loading="lazy">.
1431
1432 // Proper dimensions to avoid layout shifts.
1433
1434 // 9. What are custom data attributes, and how are they used?
1435
1436 // Custom attributes store extra info on HTML elements.
1437
1438 // Format: data-*
1439
1440 // Access via JS: element.dataset.
1441
1442 // <div data-user-id="123" data-role="admin"></div>
1443 // <script>
1444 //   const div = document.querySelector('div');
1445 //   console.log(div.dataset.userId); // 123
1446 // </script>
1447
1448
1449 // 10. How would you create an accessible custom tooltip using only HTML?
1450
1451 // Use the title attribute for basic accessibility.
```

```
1452
1453 // For custom HTML tooltips: combine aria attributes and hidden elements.
1454
1455 // <button aria-describedby="tooltip1">Hover me</button>
1456 // <div role="tooltip" id="tooltip1" hidden>
1457 //   This is an accessible tooltip.
1458 // </div>
1459
1460
1461 // Screen readers read aria-describedby.
1462
1463 // Can toggle hidden on hover/focus for visual display with CSS.
1464
1465 // If you want, I can also prepare a complete HTML + CSS + JS interview cheat sheet with
1466
1467 // Do you want me to do that?
1468
1469 // You said:
1470 // // 1. What are closures, and how are they used?
1471 // // 2. Explain the difference between var, let, and const.
1472 // // 3. What is the event loop in JavaScript?
1473 // // 4. How does prototypal inheritance work?
1474 // // 5. What are promises, and how do they compare to async/await?
1475 // // 6. What is the difference between == and ===?
1476 // // 7. Explain the concept of "hoisting" in JavaScript.
1477 // // 8. How do you handle errors in JavaScript?
1478 // // 9. What is debouncing and throttling in JavaScript?
1479 // // 10. How would you implement a deep clone of an object without using libraries?
1480 // ChatGPT said:
1481
1482 // Here's a detailed explanation for your advanced JavaScript questions:
1483
1484 // 1. What are closures, and how are they used?
1485
1486 // Closure: A function that remembers variables from its outer scope even after the outer
1487
1488 // Use cases: Encapsulation, private variables, function factories.
1489
1490 // function outer() {
1491 //   let count = 0;
1492 //   return function inner() {
1493 //     count++;
1494 //     console.log(count);
1495 //   }
1496 // }
1497 // const counter = outer();
1498 // counter(); // 1
1499 // counter(); // 2
1500
1501
1502 // 2. Difference between var, let, and const:
1503
```

```
1504 // Feature var let const
1505 // Scope Function Block Block
1506 // Hoisting Yes (initialized as undefined) Yes (TDZ) Yes (TDZ)
1507 // Reassignable Yes Yes No
1508 // Redeclarable Yes No No
1509
1510 // 3. What is the event loop in JavaScript?
1511
1512 // The event loop handles asynchronous operations in JS.
1513
1514 // JS is single-threaded. The event loop checks the call stack and task queue, executing
1515
1516 // console.log("Start");
1517 // setTimeout(() => console.log("Timeout"), 0);
1518 // console.log("End");
1519 // // Output: Start → End → Timeout
1520
1521
1522 // 4. How does prototypal inheritance work?
1523
1524 // Objects inherit properties and methods from a prototype object.
1525
1526 // Example:
1527
1528 // function Person(name) { this.name = name; }
1529 // Person.prototype.greet = function() { console.log(`Hello ${this.name}`); };
1530 // const p = new Person("Sai");
1531 // p.greet(); // Hello Sai
1532
1533
1534 // p accesses greet via its prototype chain.
1535
1536 // 5. What are promises, and how do they compare to async/await?
1537
1538 // Promise: Represents a value that may be available now, later, or never.
1539
1540 // const promise = new Promise((resolve, reject) => {
1541 //   setTimeout(() => resolve("Done!"), 1000);
1542 // });
1543 // promise.then(console.log);
1544
1545
1546 // async/await: Syntactic sugar over promises for cleaner asynchronous code.
1547
1548 // async function fetchData() {
1549 //   const result = await promise;
1550 //   console.log(result);
1551 // }
1552
1553
1554 // 6. Difference between == and ===:
1555
1556 // Operator Comparison Type Conversion
```

```
1557 // == Value equality Performs type coercion
1558 // === Value + type equality No coercion
1559 // 5 == "5"; // true
1560 // 5 === "5"; // false
1561
1562
1563 // 7. Explain the concept of hoisting in JavaScript:
1564
1565 // JS moves variable and function declarations to the top of their scope during compilation
1566
1567 // console.log(a); // undefined
1568 // var a = 5;
1569
1570
1571 // let and const are hoisted but cannot be accessed before declaration (Temporal Dead Zone)
1572
1573 // 8. How do you handle errors in JavaScript?
1574
1575 // Using try...catch...finally for synchronous code.
1576
1577 // For promises: .catch() or try/catch with async/await.
1578
1579 // try {
1580 //   let x = y; // ReferenceError
1581 // } catch (error) {
1582 //   console.log(error.message);
1583 // } finally {
1584 //   console.log("Execution complete");
1585 // }
1586
1587
1588 // 9. What is debouncing and throttling in JavaScript?
1589
1590 // Debouncing: Executes a function after a delay since the last call.
1591
1592 // Throttling: Executes a function at most once every interval, no matter how many times
1593
1594 // // Debounce
1595 // function debounce(fn, delay) {
1596 //   let timer;
1597 //   return (...args) => {
1598 //     clearTimeout(timer);
1599 //     timer = setTimeout(() => fn(...args), delay);
1600 //   }
1601 // }
1602
1603 // // Throttle
1604 // function throttle(fn, limit) {
1605 //   let lastCall = 0;
1606 //   return (...args) => {
1607 //     const now = Date.now();
1608 //     if (now - lastCall >= limit) {
```

```

1609     //     lastCall = now;
1610     //     fn(...args);
1611     // }
1612     // }
1613     // }
1614
1615
1616     // 10. How would you implement a deep clone of an object without using libraries?
1617
1618     // Using recursion or JSON.parse(JSON.stringify(obj)) (simple objects).
1619
1620     // function deepClone(obj) {
1621     //   if (obj === null || typeof obj !== "object") return obj;
1622     //   if (Array.isArray(obj)) return obj.map(deepClone);
1623     //   const copy = {};
1624     //   for (let key in obj) {
1625     //     if (obj.hasOwnProperty(key)) copy[key] = deepClone(obj[key]);
1626     //   }
1627     //   return copy;
1628     // }
1629     // const original = { a: 1, b: { c: 2 } };
1630     // const clone = deepClone(original);
1631
1632
1633     // If you want, I can create a single ultimate JavaScript + React + HTML/CSS interview c
1634
1635     // Do you want me to do that?
1636
1637     // You said:
1638     // // 1. What is the virtual DOM, and how does it work?
1639     // // 2. How do React functional components differ from class components?
1640     // // 3. What are React hooks, and why are they used?
1641     // // 4. Explain the concept of lifting state up in React.
1642     // // 5. What is the purpose of useEffect and how is it used?
1643     // // 6. How does context API work in React?
1644     // // 7. What is the difference between controlled and uncontrolled components?
1645     // // 8. How do you optimize a React application's performance?
1646     // // 9. What are React portals, and how are they useful?
1647     // // 10. How would you implement a custom hook to debounce user input in a React applic
1648     // ChatGPT said:
1649
1650     // Here's a detailed explanation for your React questions:
1651
1652     // 1. What is the Virtual DOM, and how does it work?
1653
1654     // Virtual DOM is a lightweight copy of the real DOM in memory.
1655
1656     // React diffs the virtual DOM with the previous virtual DOM (reconciliation) and only u
1657
1658     // DOM Update:
1659     // Old Virtual DOM → Compare → Patch → Real DOM update
1660
1661

```

```
1662 // 2. How do React functional components differ from class components?
1663
1664 // Feature    Functional Component    Class Component
1665 // Syntax     JS function ES6 class
1666 // State      useState hook    this.state
1667 // Lifecycle   useEffect hook    Lifecycle methods
1668 // this keyword Not used    Required to access state/methods
1669 // Performance Slightly better, simpler    Slightly heavier, verbose
1670
1671 // 3. What are React hooks, and why are they used?
1672
1673 // Hooks are functions that let you use state and other React features in functional components
1674
1675 // Common hooks: useState, useEffect, useMemo, useCallback.
1676
1677 // Benefits:
1678
1679 // Avoids class components.
1680
1681 // Reusable logic.
1682
1683 // Cleaner and simpler code.
1684
1685 // 4. Explain the concept of lifting state up in React.
1686
1687 // When multiple components need shared state, move the state to the closest common parent component
1688
1689 // The parent passes the state and updater functions as props.
1690
1691 // function Parent() {
1692 //   const [value, setValue] = useState('');
1693 //   return <><ChildA value={value} setValue={setValue} /><ChildB value={value} /></>;
1694 // }
1695
1696
1697 // 5. What is the purpose of useEffect and how is it used?
1698
1699 // useEffect handles side effects like API calls, subscriptions, or DOM manipulations in functional components
1700
1701 // useEffect(() => {
1702 //   console.log("Component mounted or updated");
1703 //   return () => console.log("Cleanup before unmount or next effect");
1704 // }, [dependencies]); // [] = run once, [var] = run when var changes
1705
1706
1707 // 6. How does Context API work in React?
1708
1709 // Context API allows passing data through the component tree without props drilling.
1710
1711 // Create a context, provide a value, consume with useContext.
1712
1713 // const ThemeContext = React.createContext('light');
```

```
1714 // <ThemeContext.Provider value="dark">
1715 //   <Child />
1716 // </ThemeContext.Provider>
1717 // const theme = useContext(ThemeContext);
1718
1719
1720 // 7. Difference between controlled and uncontrolled components:
1721
1722 // Feature   Controlled   Uncontrolled
1723 // State     React state   DOM state
1724 // Input value   Controlled by value prop   Managed by DOM, accessed via ref
1725 // Example   <input value={state} onChange={handleChange} /> <input ref={inputRef} />
1726
1727 // 8. How do you optimize a React application's performance?
1728
1729 // Memoization: React.memo, useMemo, useCallback.
1730
1731 // Lazy loading components with React.lazy and Suspense.
1732
1733 // Avoid unnecessary re-renders.
1734
1735 // Code splitting.
1736
1737 // Optimize images and assets.
1738
1739 // Use efficient data structures and algorithms.
1740
1741 // 9. What are React portals, and how are they useful?
1742
1743 // Portals allow rendering a component outside its parent DOM hierarchy.
1744
1745 // Useful for modals, tooltips, or overlays.
1746
1747 // ReactDOM.createPortal(<Modal />, document.getElementById('modal-root'));
1748
1749
1750 // 10. How would you implement a custom hook to debounce user input in React?
1751
1752 // import { useState, useEffect } from "react";
1753
1754 // function useDebounce(value, delay) {
1755 //   const [debouncedValue, setDebouncedValue] = useState(value);
1756 //   useEffect(() => {
1757 //     const handler = setTimeout(() => setDebouncedValue(value), delay);
1758 //     return () => clearTimeout(handler); // cleanup
1759 //   }, [value, delay]);
1760 //   return debouncedValue;
1761 // }
1762
1763 // // Usage
1764 // const [search, setSearch] = useState('');
1765 // const debouncedSearch = useDebounce(search, 500);
```

```
1/66
1767
1768 // The hook delays updating debouncedSearch until the user stops typing for 500ms.
1769
1770 // If you want, I can combine all your React + JavaScript + HTML/CSS questions into a si
1771
1772 // Do you want me to create that?
1773
1774 // You said:
1775 // // 1. What is ARIA, and why is it important?
1776 // // 2. How would you ensure a form is accessible?
1777 // // 3. What are some common accessibility issues in web development?
1778 // // 4. How do you make images accessible?
1779 // // 5. What is a screen reader, and how does it work?
1780 // // 6. How would you create an accessible navigation menu?
1781 // // 7. What is the difference between role="button" and a <button> element?
1782 // // 8. How do you test a webpage for accessibility?
1783 // ChatGPT said:
1784
1785 // Here's a detailed explanation for your accessibility (a11y) questions:
1786
1787 // 1. What is ARIA, and why is it important?
1788
1789 // ARIA (Accessible Rich Internet Applications) provides attributes to make web content
1790
1791 // Adds semantic meaning to non-semantic elements for assistive technologies (like scree
1792
1793 // Example: role="dialog", aria-label="Close menu".
1794
1795 // 2. How would you ensure a form is accessible?
1796
1797 // Use <label> elements for inputs.
1798
1799 // Group related inputs with <fieldset> and <legend>.
1800
1801 // Ensure keyboard navigation (tabindex).
1802
1803 // Provide clear error messages with ARIA roles or aria-describedby.
1804
1805 // <label for="email">Email:</label>
1806 // <input type="email" id="email" aria-required="true">
1807 // <span id="emailError" role="alert">Invalid email</span>
1808
1809
1810 // 3. Common accessibility issues in web development:
1811
1812 // Missing alt text for images.
1813
1814 // Poor color contrast.
1815
1816 // Non-semantic HTML.
1817
1818 // Inaccessible forms (missing labels).
```

```
1819
1820 // Keyboard navigation issues.
1821
1822 // Improper use of ARIA roles.
1823
1824 // 4. How do you make images accessible?
1825
1826 // Use descriptive alt attributes.
1827
1828 // For decorative images, use alt="".
1829
1830 // Use captions or surrounding context when needed.
1831
1832 // 
1833
1834
1835 // 5. What is a screen reader, and how does it work?
1836
1837 // A screen reader is software that reads out webpage content to visually impaired users
1838
1839 // Works by interpreting HTML, ARIA roles, and semantic tags.
1840
1841 // Examples: NVDA, JAWS, VoiceOver.
1842
1843 // 6. How would you create an accessible navigation menu?
1844
1845 // Use semantic <nav> element.
1846
1847 // Ensure all links are focusable (tabindex).
1848
1849 // For dropdowns, use proper ARIA attributes:
1850
1851 // <nav>
1852 //   <ul>
1853 //     <li><a href="#home">Home</a></li>
1854 //     <li>
1855 //       <button aria-expanded="false" aria-controls="submenu">Services</button>
1856 //       <ul id="submenu">
1857 //         <li><a href="#web">Web Development</a></li>
1858 //       </ul>
1859 //     </li>
1860 //   </ul>
1861 // </nav>
1862
1863
1864 // 7. Difference between role="button" and a <button> element:
1865
1866 // Feature   <button>   role="button"
1867 // Native behavior   Has keyboard support & accessibility   Only adds semantic role; need
1868 // Recommended   ✅   Use only when styling non-button elements (like <div>)
1869
1870 // 8. How do you test a webpage for accessibility?
```

```
1871
1872 // Manual testing: keyboard navigation, screen readers.
1873
1874 // Automated tools:
1875
1876 // Lighthouse (Chrome)
1877
1878 // axe DevTools
1879
1880 // WAVE accessibility tool
1881
1882 // Check color contrast: WCAG guidelines.
1883
1884 // Validate semantic HTML and ARIA roles.
1885
1886 // If you want, I can create a complete "Frontend Interview Cheat Sheet" covering JavaSc
1887
1888 // Do you want me to do that?
1889
1890 // You said:
1891 // // 1. What is lazy loading, and how do you implement it?
1892 // // 2. How do you reduce the size of a JavaScript bundle?
1893 // // 3. What are the benefits of using a Content Delivery Network (CDN)?
1894 // // 4. How do you optimize CSS delivery?
1895 // // 5. What is caching, and how does it improve performance?
1896 // // 6. What are critical rendering paths?
1897 // // 7. How do you optimize for Time to First Byte (TTFB)?
1898 // // 8. How do service workers improve web performance?
1899 // ChatGPT said:
1900
1901 // Here's a detailed explanation for your web performance optimization questions:
1902
1903 // 1. What is lazy loading, and how do you implement it?
1904
1905 // Lazy loading delays loading of resources (images, scripts, or components) until they
1906
1907 // Benefits: Reduces initial load time, improves performance.
1908
1909 // Implementation in React:
1910
1911 // import React, { Suspense } from 'react';
1912 // const LazyComponent = React.lazy(() => import('./LazyComponent'));
1913 // function App() {
1914 //   return (
1915 //     <Suspense fallback={<div>Loading...</div>}>
1916 //       <LazyComponent />
1917 //     </Suspense>
1918 //   );
1919 // }
1920
1921
1922 // In HTML for images:
```

1923

```
1223
1224 // 
1225
1226
1227 // 2. How do you reduce the size of a JavaScript bundle?
1228
1229 // Use tree shaking to remove unused code.
1230
1231 // Split code using dynamic imports / code splitting.
1232
1233 // Minify and compress JS files.
1234
1235 // Remove unused dependencies and polyfills.
1236
1237 // Use lightweight libraries.
1238
1239 // 3. Benefits of using a Content Delivery Network (CDN):
1240
1241 // Serves content from servers closest to the user → faster load.
1242
1243 // Reduces server load and bandwidth costs.
1244
1245 // Provides redundancy and high availability.
1246
1247 // Can improve SEO and global performance.
1248
1249 // 4. How do you optimize CSS delivery?
1250
1251 // Inline critical CSS for above-the-fold content.
1252
1253 // Minify CSS files.
1254
1255 // Use media queries to load CSS only when needed.
1256
1257 // Remove unused CSS (tools like PurgeCSS).
1258
1259 // Load non-critical CSS asynchronously:
1260
1261 // <link rel="preload" href="styles.css" as="style" onload="this.onload=null;this.rel='s
1262
1263
1264 // 5. What is caching, and how does it improve performance?
1265
1266 // Caching stores frequently used resources locally (browser, CDN) to avoid repeated dow
1267
1268 // Benefits: Faster load times, reduced server requests.
1269
1270 // Example headers:
1271
1272 // Cache-Control: max-age=3600
1273 // ETag: "12345"
1274
1275
```

```
1976 // 6. What are critical rendering paths?
1977
1978 // The sequence of steps the browser takes to render a page.
1979
1980 // Includes: HTML parsing → CSSOM building → JS execution → DOM rendering → painting.
1981
1982 // Optimizing this path reduces Time to First Paint (TTFP) and improves UX.
1983
1984 // 7. How do you optimize for Time to First Byte (TTFB)?
1985
1986 // Use fast servers and CDNs.
1987
1988 // Optimize backend processing and database queries.
1989
1990 // Enable server-side caching.
1991
1992 // Use HTTP/2 or CDN edge caching.
1993
1994 // 8. How do service workers improve web performance?
1995
1996 // Service workers intercept network requests to cache resources and enable offline support.
1997
1998 // Benefits:
1999
2000 // Faster page loads (cached assets).
2001
2002 // Offline capabilities.
2003
2004 // Background sync for API updates.
2005
2006 // self.addEventListener('fetch', event => {
2007 //   event.respondWith(
2008 //     caches.match(event.request).then(response => response || fetch(event.request))
2009 //   );
2010 // });
2011
2012
2013 // I can also combine all your Frontend topics—JavaScript, React, HTML, CSS, Accessibility, and Performance—into a comprehensive guide.
2014
2015 // Do you want me to create that?
2016
2017 // You said:
2018 // // ▸ What happens when you type a URL in the browser?
2019 // // ▸ How does the browser show a webpage (Critical Rendering Path)?
2020 // // ▸ What's the difference between synchronous and asynchronous scripts?
2021 // // ▸ What are CORS and Same-Origin Policy?
2022 // // ▸ How does browser caching work?
2023 // // ▸ How do you debug performance issues using Chrome DevTools?
2024 // // ▸ What is the difference between prefetching, preloading, and prerendering?
2025 // ChatGPT said:
2026
2027 // Here's a detailed explanation for your web fundamentals and performance questions:
-----
```

```
2028
2029 // 1. What happens when you type a URL in the browser?
2030
2031 // DNS lookup: Converts domain to IP address.
2032
2033 // TCP connection: Browser connects to the server (via HTTP/HTTPS).
2034
2035 // HTTP request: Browser sends a request to the server.
2036
2037 // Server response: Server sends back HTML, CSS, JS, images, etc.
2038
2039 // Browser rendering: Parses HTML, builds DOM, CSSOM, executes JS.
2040
2041 // Page display: Browser paints content on the screen.
2042
2043 // 2. How does the browser show a webpage (Critical Rendering Path)?
2044
2045 // Critical Rendering Path (CRP): Steps the browser takes to convert HTML/CSS/JS into pixels
2046
2047 // HTML parsing → DOM tree
2048
2049 // CSS parsing → CSSOM tree
2050
2051 // Render tree construction (DOM + CSSOM)
2052
2053 // Layout / Reflow: Calculates geometry.
2054
2055 // Paint: Draw pixels to screen.
2056
2057 // Composite: Combine layers for final display.
2058
2059 // Optimizing CRP improves Time to First Paint (TTFP) and Largest Contentful Paint (LCP)
2060
2061 // 3. Difference between synchronous and asynchronous scripts:
2062
2063 // Type Behavior      Example
2064 // Synchronous  Browser blocks parsing until script is executed <script src="script.js">
2065 // Asynchronous Script is downloaded in parallel and executed without blocking HTML parsing
2066 // Defer        Script downloads in parallel, executes after HTML parsing <script src="script.js" defer>
2067
2068 // 4. What are CORS and Same-Origin Policy?
2069
2070 // Same-Origin Policy: Browser restricts scripts from accessing data from a different origin
2071
2072 // CORS (Cross-Origin Resource Sharing): Server allows cross-origin requests via response headers
2073
2074 // Access-Control-Allow-Origin: https://example.com
2075
2076
2077 // 5. How does browser caching work?
2078
2079 // Browser stores resources locally to reduce network requests.
2080
```

```

2081 // Types:
2082
2083 // HTTP caching: Cache-Control, Expires, ETag.
2084
2085 // Service Worker caching: Offline-first or custom strategies.
2086
2087 // On repeat visits, browser may serve cached resources instead of downloading again.
2088
2089 // 6. How do you debug performance issues using Chrome DevTools?
2090
2091 // Network tab: Check requests, load times, and caching.
2092
2093 // Performance tab: Record page load, JavaScript execution, layout shifts.
2094
2095 // Lighthouse: Audit page speed, accessibility, SEO, best practices.
2096
2097 // Coverage: Find unused CSS and JS.
2098
2099 // Memory tab: Detect memory leaks and heap usage.
2100
2101 // 7. Difference between prefetching, preloading, and prerendering:
2102
2103 // Technique Purpose Example
2104 // Preload Load critical resources early to improve render <link rel="preload" href="st
2105 // Prefetch Load non-critical resources for future navigation <link rel="prefetch" href
2106 // Prerender Load and render a full page in background before user visits <link re
2107
2108 // Preload = needed now, prefetch = needed later, prerender = fully render page ahead.
2109
2110 // If you want, I can create a single complete Frontend Interview Cheat Sheet with 100+
2111
2112 // Do you want me to create that?
2113
2114 // You said:
2115 // // ▸ What is the difference between unit, integration, and end-to-end testing?
2116 // // ▸ What is snapshot testing?
2117 // // ▸ How would you test APIs in a frontend project?
2118 // // ▸ How do you ensure a website works in all browsers?
2119 // // ▸ What tools are used for testing accessibility?
2120 // // ▸ How do you test React components?
2121 // ChatGPT said:
2122
2123 // Here's a detailed explanation for your frontend testing questions:
2124
2125 // 1. Difference between unit, integration, and end-to-end (E2E) testing:
2126
2127 // Testing Type Purpose Scope Example
2128 // Unit testing Test individual functions/components in isolation Single function/comp
2129 // Integration testing Test interaction between modules/components Multiple units toget
2130 // End-to-end (E2E) testing Test full application flow from user perspective Whole ap
2131
2132 // 2. What is snapshot testing?

```

```
2133
2134 // Snapshot testing captures a rendered output of a component and compares it to a saved
2135
2136 // Useful to detect unexpected changes in UI.
2137
2138 // Tools: Jest + React Testing Library
2139
2140 // import { render } from '@testing-library/react';
2141 // import Button from './Button';
2142
2143 // test('matches snapshot', () => {
2144 //   const { asFragment } = render(<Button label="Click" />);
2145 //   expect(asFragment()).toMatchSnapshot();
2146 // });
2147
2148
2149 // 3. How would you test APIs in a frontend project?
2150
2151 // Manual testing: Postman, Insomnia.
2152
2153 // Automated testing: Jest + Axios or fetch mocks.
2154
2155 // import axios from 'axios';
2156 // jest.mock('axios');
2157
2158 // test('fetches data from API', async () => {
2159 //   axios.get.mockResolvedValue({ data: { id: 1, name: 'Test' } });
2160 //   const response = await fetchData();
2161 //   expect(response.name).toBe('Test');
2162 // });
2163
2164
2165 // 4. How do you ensure a website works in all browsers?
2166
2167 // Use cross-browser testing tools: BrowserStack, LambdaTest.
2168
2169 // Test on major browsers (Chrome, Firefox, Safari, Edge).
2170
2171 // Use vendor prefixes in CSS or autoprefixer.
2172
2173 // Use polyfills for unsupported JS features (Babel).
2174
2175 // Follow progressive enhancement and feature detection.
2176
2177 // 5. Tools used for testing accessibility:
2178
2179 // Automated tools: Lighthouse, axe DevTools, WAVE, Pa11y.
2180
2181 // Manual testing: Keyboard navigation, screen readers (NVDA, VoiceOver, JAWS).
2182
2183 // Linting: eslint-plugin-jsx-a11y for React projects.
2184
2185 // 6. How do you test React components?
```

```

2185 // 6. How do you test React components?
2186
2187 // Unit testing: Jest + React Testing Library / Enzyme.
2188
2189 // Snapshot testing: Verify UI changes.
2190
2191 // Simulate events: Test user interactions with fireEvent or userEvent.
2192
2193 // import { render, fireEvent } from '@testing-library/react';
2194 // import Counter from './Counter';
2195
2196 // test('increments counter', () => {
2197 //   const { getByText } = render(<Counter />);
2198 //   fireEvent.click(getByText('Increment'));
2199 //   expect(getByText('Count: 1')).toBeInTheDocument();
2200 // });
2201
2202
2203 // If you want, I can compile all your frontend topics (JavaScript, React, HTML/CSS, Per
2204
2205 // Do you want me to create that?
2206
2207 // You said:
2208 // // ▸ What is a micro-frontend?
2209 // // ▸ How does a Single Page Application (SPA) differ from a Multi-Page Application (M
2210 // // ▸ How does server-side rendering (SSR) compare to client-side rendering (CSR)?
2211 // // ▸ What are the benefits of modular or reusable components?
2212 // ChatGPT said:
2213
2214 // Here's a detailed explanation for your frontend architecture and best practices quest
2215
2216 // 1. What is a micro-frontend?
2217
2218 // Micro-frontend is an architecture pattern where a frontend app is split into smaller,
2219
2220 // Each micro-frontend can be developed and deployed by different teams.
2221
2222 // Benefits: scalability, independent releases, and easier maintenance.
2223
2224 // Example: A large e-commerce site with separate micro-frontends for cart, product list
2225
2226 // 2. How does a Single Page Application (SPA) differ from a Multi-Page Application (MPA
2227
2228 // Feature   SPA   MPA
2229 // Page reload   No full reload, only dynamic content updates   Full page reload for each
2230 // URL routing   Handled client-side (React Router, Vue Router)   Handled server-side
2231 // Performance   Faster navigation after initial load   Slower due to multiple full page
2232 // SEO           Needs extra setup (SSR or prerendering)   Naturally SEO-friendly
2233 // Example       React, Angular, Vue apps   Traditional PHP, Java, or .NET apps
2234
2235 // 3. How does server-side rendering (SSR) compare to client-side rendering (CSR)?
2236
2237 // Feature   SSR   CSR

```

```
2238 // Rendering    HTML generated on the server    HTML generated in the browser
2239 // First load    Faster (content visible immediately)    Slower initial load
2240 // SEO    Good (search engines see content)    Poor without SSR/prerendering
2241 // Interaction    Requires more client JS for dynamic behavior    Fully handled by client
2242 // Example    Next.js, Nuxt.js    React SPA, Vue SPA
2243
2244 // 4. Benefits of modular or reusable components:
2245
2246 // Reusability: Write once, use in multiple places.
2247
2248 // Maintainability: Isolated changes don't affect unrelated components.
2249
2250 // Testability: Smaller units are easier to test.
2251
2252 // Consistency: Uniform design and behavior across the app.
2253
2254 // Scalability: Makes large applications easier to manage and extend.
2255
2256 // If you want, I can combine all your Frontend questions—from JS, React, HTML/CSS, Perf
2257
2258 // Do you want me to create that?
2259
2260 // You said:
2261 // // ▸ Difference between picture and img elements?
2262 // // ▸ Purpose of object tag vs. iframe?
2263 // // ▸ What are ARIA attributes and their importance?
2264 // // ▸ How do you implement a responsive grid in HTML5?
2265 // // ▸ localStorage vs. sessionStorage?
2266 // // ▸ What are Web Components and how to create one?
2267 // // ▸ How does Shadow DOM work?
2268 // // ▸ What are Web Workers and their benefits?
2269 // // ▸ What is contenteditable attribute?
2270 // // ▸ canvas vs. SVG for graphics?
2271 // ChatGPT said:
2272
2273 // Here's a detailed explanation for your advanced HTML5 and web APIs questions:
2274
2275 // 1. Difference between <picture> and <img> elements:
2276
2277 // <img>: Standard image element, single source.
2278
2279 // <picture>: Allows responsive images and multiple sources (e.g., WebP fallback, differ
2280
2281 // <picture>
2282 //   <source srcset="image.webp" type="image/webp">
2283 //   
2284 // </picture>
2285
2286
2287 // 2. Purpose of <object> tag vs. <iframe>:
2288
2289 // Tag    Purpose Notes
```

```
2290 // <object> Embeds external content (HTML, images, PDFs, multimedia) Can fallback cor
2291 // <iframe> Embeds another HTML document inside a page Isolated browsing context
2292
2293 // 3. What are ARIA attributes and their importance?
2294
2295 // ARIA (Accessible Rich Internet Applications) attributes enhance accessibility for scr
2296
2297 // Examples: role, aria-label, aria-expanded, aria-hidden.
2298
2299 // Importance: Improves a11y, ensures dynamic content is accessible.
2300
2301 // 4. How do you implement a responsive grid in HTML5?
2302
2303 // Using CSS Grid or Flexbox with media queries:
2304
2305 // <div class="grid">
2306 //   <div>1</div><div>2</div><div>3</div>
2307 // </div>
2308 // <style>
2309 //   .grid {
2310 //     display: grid;
2311 //     grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
2312 //     gap: 10px;
2313 //   }
2314 // </style>
2315
2316
2317 // 5. localStorage vs. sessionStorage:
2318
2319 // Feature   localStorage   sessionStorage
2320 // Lifespan  Persistent   Until tab/browser closes
2321 // Storage limit   ~5-10MB   ~5MB
2322 // Scope      Shared across all tabs of same origin   Tab-specific
2323 // API        setItem, getItem, removeItem   Same
2324
2325 // 6. What are Web Components and how to create one?
2326
2327 // Web Components: Encapsulated, reusable custom HTML elements.
2328
2329 // Created using: Custom Elements, Shadow DOM, HTML Templates
2330
2331 // class MyComponent extends HTMLElement {
2332 //   connectedCallback() {
2333 //     this.innerHTML = "<p>Hello Web Component</p>";
2334 //   }
2335 // }
2336 // customElements.define('my-component', MyComponent);
2337
2338
2339 // 7. How does Shadow DOM work?
2340
2341 // Shadow DOM provides encapsulation for DOM and CSS.
2342
```

```
2343 // Prevents styles from leaking in/out.
2344
2345 // const shadow = document.querySelector('#host').attachShadow({mode: 'open'});
2346 // shadow.innerHTML = `

Shadow content</p>`;
2347
2348
2349 // 8. What are Web Workers and their benefits?
2350
2351 // Web Workers run JavaScript in a background thread, preventing UI blocking.
2352
2353 // Benefits: Smooth UI, heavy computations offloaded.
2354
2355 // const worker = new Worker('worker.js');
2356 // worker.postMessage('data');
2357 // worker.onmessage = (e) => console.log(e.data);
2358
2359
2360 // 9. What is contenteditable attribute?
2361
2362 // Makes an element editable in the browser.
2363
2364 // <div contenteditable="true">Edit me</div>
2365
2366
2367 // 10. canvas vs. SVG for graphics:
2368
2369 // Feature canvas SVG
2370 // Type Raster (pixel-based) Vector (scalable)
2371 // Use case Complex animations, game graphics Charts, icons, illustrations
2372 // DOM integration Not part of DOM Part of DOM, accessible
2373 // Performance Better for many dynamic objects Better for few static/scalable objects
2374
2375 // If you want, I can compile all your Frontend topics—HTML, CSS, JS, React, Performance
2376
2377 // Do you want me to create that?
2378
2379 // You said:
2380 // // ▸ How does this work in different contexts?
2381 // // ▸ Shallow vs. deep copy in JS?
2382 // // ▸ What is the Module Pattern in JS?
2383 // // ▸ How do callbacks, promises, and async/await affect the event loop?
2384 // // ▸ What are decorators in JS?
2385 // // ▸ Difference between call(), apply(), bind()?
2386 // // ▸ What is the Reflect API in JS?
2387 // // ▸ How to handle race conditions in JS?
2388 // // ▸ What is WeakMap vs. Map?
2389 // // ▸ Explain event delegation in JS.
2390 // ChatGPT said:
2391
2392 // Here's a detailed explanation for your advanced JavaScript questions:
2393
2394 // 1. How does this work in different contexts?


```

```
2395
2396 // Context this value
2397 // Global scope window (browser) / undefined (strict mode)
2398 // Function window / undefined (strict mode)
2399 // Object method The object calling the method
2400 // Constructor (new) Newly created object
2401 // Arrow function Lexical this (inherits from surrounding scope)
2402 // const obj = {
2403 //   a: 10,
2404 //   method: function() { console.log(this.a); }
2405 // };
2406 // obj.method(); // 10
2407
2408
2409 // 2. Shallow vs. Deep copy in JS:
2410
2411 // Shallow copy: Copies only top-level properties; nested objects are still referenced.
2412
2413 // Deep copy: Recursively copies all nested objects, no references to original.
2414
2415 // const obj = { a: 1, b: { c: 2 } };
2416 // const shallow = {...obj};
2417 // const deep = JSON.parse(JSON.stringify(obj));
2418
2419
2420 // 3. What is the Module Pattern in JS?
2421
2422 // Encapsulates code into private and public parts using closures.
2423
2424 // Prevents global namespace pollution.
2425
2426 // const Module = (function() {
2427 //   let privateVar = 0;
2428 //   return {
2429 //     increment() { privateVar++; },
2430 //     getValue() { return privateVar; }
2431 //   };
2432 // })();
2433 // Module.increment();
2434 // console.log(Module.getValue()); // 1
2435
2436
2437 // 4. How do callbacks, promises, and async/await affect the event loop?
2438
2439 // Callbacks: Added to task queue after main thread execution.
2440
2441 // Promises: .then callbacks go to microtask queue, executed before task queue.
2442
2443 // async/await: Syntax sugar over promises, still uses microtask queue.
2444
2445 // console.log('Start');
2446 // setTimeout(() => console.log('Timeout'), 0);
2447 // Promise.resolve().then(() => console.log('Promise'));
```

```
2447 // Promise.resolve().then() → console.log('Promise');  
2448 // console.log('End');  
2449 // // Output: Start → End → Promise → Timeout  
2450  
2451  
2452 // 5. What are decorators in JS?  
2453  
2454 // Functions that wrap or modify classes/methods/properties.  
2455  
2456 // Common in frameworks like Angular or TypeScript.  
2457  
2458 // function readonly(target, key, descriptor) {  
2459 //   descriptor.writable = false;  
2460 //   return descriptor;  
2461 // }  
2462 // class Example {  
2463 //   @readonly  
2464 //   name = "Sai";  
2465 // }  
2466  
2467  
2468 // 6. Difference between call(), apply(), and bind():  
2469  
2470 // Method    Purpose  
2471 // call(thisArg, arg1, arg2)    Invokes function with given this and arguments  
2472 // apply(thisArg, [args])    Same as call, but takes arguments as array  
2473 // bind(thisArg, arg1, arg2)    Returns a new function with bound this and arguments  
2474  
2475 // 7. What is the Reflect API in JS?  
2476  
2477 // Provides methods for interceptable object operations (like proxy traps).  
2478  
2479 // Examples: Reflect.get, Reflect.set, Reflect.has  
2480  
2481 // const obj = {a: 1};  
2482 // console.log(Reflect.get(obj, 'a')); // 1  
2483  
2484  
2485 // 8. How to handle race conditions in JS?  
2486  
2487 // Avoid shared mutable state.  
2488  
2489 // Use Promises / async/await sequencing.  
2490  
2491 // Example: Chain async operations instead of running concurrently if order matters.  
2492  
2493 // await firstAsync();  
2494 // await secondAsync(); // Ensures correct sequence  
2495  
2496  
2497 // 9. What is WeakMap vs. Map?  
2498  
2499 // Feature    Map    WeakMap
```

```

2500 // Keys Any type    Only objects
2501 // Garbage collection    Keys prevent GC Keys do not prevent GC (weak reference)
2502 // Iteration    Iterable    Not iterable
2503 // Use case General key-value storage    Private data for objects
2504
2505 // 10. Explain event delegation in JS:
2506
2507 // Event delegation attaches a single event listener to a parent element and leverages e
2508
2509 // Efficient for dynamically added elements.
2510
2511 // document.querySelector('#parent').addEventListener('click', (e) => {
2512 //   if(e.target.tagName === 'BUTTON') console.log('Button clicked');
2513 // });
2514
2515
2516 // If you want, I can combine all your JS, React, HTML, CSS, Performance, Accessibility,
2517
2518 // Do you want me to create that?
2519
2520 // You said:
2521 // // ▸ useEffect vs. useLayoutEffect?
2522 // // ▸ React lifecycle methods in class vs. functional components?
2523 // // ▸ Managing global state without Redux/Context API?
2524 // // ▸ How does React's reconciliation work?
2525 // // ▸ Implementing SSR and SSG in React?
2526 // // ▸ What are Suspense and Lazy loading in React?
2527 // // ▸ Error boundaries in React?
2528 // // ▸ What is React strict mode?
2529 // // ▸ React memoization: React.memo(), useMemo(), useCallback()?
2530 // // ▸ What is React Fiber architecture?
2531 // ChatGPT said:
2532
2533 // Here's a detailed explanation for your advanced React questions:
2534
2535 // 1. useEffect vs. useLayoutEffect:
2536
2537 // Feature    useEffect    useLayoutEffect
2538 // Timing    Runs after painting (non-blocking)    Runs before painting (blocking)
2539 // Use case    Fetching data, subscriptions    Measuring DOM, synchronously manipulating la
2540 // Performance    Doesn't block UI    Can block rendering if heavy
2541 // useLayoutEffect(() => {
2542 //   console.log('DOM updated, before paint');
2543 // }, []);
2544 // useEffect(() => {
2545 //   console.log('After paint');
2546 // }, []);
2547
2548
2549 // 2. React lifecycle methods: class vs. functional components:
2550
2551 // Phase    Class Component    Functional Component

```

```
2552 // Mount    constructor → render → componentDidMount    useEffect(() => {}, [])
2553 // Update    shouldComponentUpdate → render → componentDidUpdate    useEffect(() => {}, [dep
2554 // Unmount    componentWillUnmount    useEffect(() => return cleanup, [])
2555
2556 // 3. Managing global state without Redux/Context API:
2557
2558 // Use lifting state up to common parent.
2559
2560 // Use custom hooks with shared state.
2561
2562 // Third-party libraries: Zustand, Jotai, Recoil.
2563
2564 // 4. How does React's reconciliation work?
2565
2566 // React uses a diffing algorithm to update the Virtual DOM efficiently.
2567
2568 // Steps:
2569
2570 // Compare new virtual DOM with previous.
2571
2572 // Find minimum changes.
2573
2574 // Update only the changed parts of the real DOM.
2575
2576 // Uses keys in lists to identify elements and avoid unnecessary re-renders.
2577
2578 // 5. Implementing SSR and SSG in React:
2579
2580 // SSR (Server-Side Rendering): HTML is generated on the server on each request.
2581
2582 // Example: Next.js getServerSideProps().
2583
2584 // SSG (Static Site Generation): HTML is generated at build time.
2585
2586 // Example: Next.js getStaticProps().
2587
2588 // 6. Suspense and Lazy loading in React:
2589
2590 // Lazy loading: Load components on demand to reduce bundle size.
2591
2592 // Suspense: Shows a fallback while a lazy component is loading.
2593
2594 // const LazyComp = React.lazy(() => import('./LazyComp'));
2595 // <Suspense fallback={<div>Loading...</div>}>
2596 //   <LazyComp />
2597 // </Suspense>
2598
2599
2600 // 7. Error boundaries in React:
2601
2602 // Components that catch JavaScript errors in their child tree and render a fallback UI.
2603
2604 // Only class components can be error boundaries.
```

```
2605
2606 // class ErrorBoundary extends React.Component {
2607 //   state = { hasError: false };
2608 //   static getDerivedStateFromError() { return { hasError: true }; }
2609 //   render() { return this.state.hasError ? <h1>Error occurred</h1> : this.props.children; }
2610 // }
2611
2612
2613 // 8. React Strict Mode:
2614
2615 // Development-only tool to highlight potential problems (deprecated APIs, side effects)
2616
2617 // Does double-invocation of certain functions to detect issues.
2618
2619 // <React.StrictMode>
2620 //   <App />
2621 // </React.StrictMode>
2622
2623
2624 // 9. React memoization: React.memo(), useMemo(), useCallback():
2625
2626 // Method    Purpose
2627 // React.memo(Component)    Memoizes functional component rendering based on props
2628 // useMemo(() => compute, [deps])    Memoizes computed values
2629 // useCallback(() => fn, [deps])    Memoizes functions to prevent re-creation
2630
2631 // 10. React Fiber architecture:
2632
2633 // Fiber is React's reconciliation engine introduced in React 16.
2634
2635 // Enables:
2636
2637 // Incremental rendering (pause, resume work).
2638
2639 // Prioritized updates.
2640
2641 // Better handling of asynchronous rendering.
2642
2643 // Improves responsiveness for large component trees.
2644
2645 // If you want, I can compile all your React + JS + HTML/CSS + Performance + Accessibility questions.
2646
2647 // Do you want me to create that?
2648
2649 // You said:
2650 // // ▸ How to create an accessible custom dropdown?
2651 // // ▸ What is aria-live and its use?
2652 // // ▸ Focus management in SPAs?
2653 // // ▸ Use of aria-hidden for dynamic content?
2654 // // ▸ Making tables accessible to screen readers?
2655 // // ▸ Explain tabindex and focus order in forms?
2656 // // ▸ Accessible multimedia elements?
```

```
2657 // // ▸ Best practices for accessible form error messages?
2658 // // ▸ Testing accessibility: automated vs. manual methods?
2659 // // ▸ accesskey attribute in accessibility?
2660 // ChatGPT said:
2661
2662 // Here's a detailed explanation for your accessibility (a11y) questions in frontend dev
2663
2664 // 1. How to create an accessible custom dropdown?
2665
2666 // Use semantic roles and keyboard navigation.
2667
2668 // Example:
2669
2670 // <div role="combobox" aria-expanded="false" tabindex="0" id="dropdown">
2671 //   <span>Choose an option</span>
2672 //   <ul role="listbox" aria-labelledby="dropdown" hidden>
2673 //     <li role="option" tabindex="-1">Option 1</li>
2674 //     <li role="option" tabindex="-1">Option 2</li>
2675 //   </ul>
2676 // </div>
2677
2678
2679 // Ensure arrow keys navigate options, Enter selects, Esc closes.
2680
2681 // 2. What is aria-live and its use?
2682
2683 // aria-live notifies screen readers of dynamic content changes.
2684
2685 // Values: polite, assertive, off.
2686
2687 // <div aria-live="polite" id="notification">New message received</div>
2688
2689
2690 // 3. Focus management in SPAs
2691
2692 // SPAs don't reload pages; focus must move to updated content.
2693
2694 // Best practices:
2695
2696 // Set focus to main heading or modal when route changes.
2697
2698 // Use tabindex="-1" for programmatic focus.
2699
2700 // document.getElementById('main-heading').focus();
2701
2702
2703 // 4. Use of aria-hidden for dynamic content
2704
2705 // aria-hidden="true" hides non-visible elements from screen readers.
2706
2707 // Example: Hiding background content when modal is open.
2708
2709 // <div aria-hidden="true">Background content</div>
```

```

2709 // <div aria-hidden="true">Background and content</div>
2710
2711
2712 // 5. Making tables accessible to screen readers
2713
2714 // Use <thead>, <tbody>, <th> for headers.
2715
2716 // Use scope="col" / scope="row" for clarity.
2717
2718 // Example:
2719
2720 // <table>
2721 //   <thead>
2722 //     <tr><th scope="col">Name</th><th scope="col">Age</th></tr>
2723 //   </thead>
2724 //   <tbody>
2725 //     <tr><td>John</td><td>25</td></tr>
2726 //   </tbody>
2727 // </table>
2728
2729
2730 // 6. Explain tabindex and focus order in forms
2731
2732 // tabindex controls keyboard focus order.
2733
2734 // 0 → follows natural DOM order.
2735
2736 // -1 → focusable programmatically only.
2737
2738 // >0 → custom tab order (use sparingly).
2739
2740 // <input tabindex="2">
2741 // <input tabindex="1">
2742
2743
2744 // 7. Accessible multimedia elements
2745
2746 // Videos: Provide captions, transcripts, audio descriptions.
2747
2748 // Audio: Provide transcripts.
2749
2750 // Use <track> for subtitles:
2751
2752 // <video controls>
2753 //   <source src="movie.mp4" type="video/mp4">
2754 //   <track src="subs.vtt" kind="subtitles" srclang="en" label="English">
2755 // </video>
2756
2757
2758 // 8. Best practices for accessible form error messages
2759
2760 // Associate errors with inputs using aria-describedby.
2761

```

```
2762 // Provide clear and descriptive messages.
2763
2764 // <input id="email" aria-describedby="emailError">
2765 // <span id="emailError" role="alert">Enter a valid email</span>
2766
2767
2768 // 9. Testing accessibility: automated vs. manual methods
2769
2770 // Automated tools: Lighthouse, axe DevTools, WAVE.
2771
2772 // Manual testing:
2773
2774 // Keyboard navigation (Tab, Shift+Tab).
2775
2776 // Screen reader testing (NVDA, JAWS, VoiceOver).
2777
2778 // Color contrast and focus visibility.
2779
2780 // 10. accesskey attribute in accessibility
2781
2782 // Assign keyboard shortcuts to elements.
2783
2784 // Example:
2785
2786 // <button accesskey="s">Save</button>
2787
2788
2789 // Use carefully to avoid conflicts with browser shortcuts.
2790
2791 // If you want, I can compile all your accessibility, React, JS, HTML, CSS, Performance,
2792
2793 // Do you want me to create that?
2794
2795 // You said:
2796 // // ▸ Difference between HTTP/1.1 and HTTP/2?
2797 // // ▸ How to implement Critical CSS?
2798 // // ▸ Efficient caching strategies for performance?
2799 // // ▸ Preventing layout shifts (CLS)?
2800 // // ▸ What is resource prefetching?
2801 // // ▸ Using Intersection Observer for lazy loading?
2802 // // ▸ Code-splitting in React for performance?
2803 // // ▸ Prioritizing critical resources for page load?
2804 // // ▸ Optimizing performance of large image galleries?
2805 // // ▸ What are Web Vitals and how to optimize Core Web Vitals?
2806 // ChatGPT said:
2807
2808 // Here's a detailed explanation for your frontend performance and optimization question
2809
2810 // 1. Difference between HTTP/1.1 and HTTP/2:
2811
2812 // Feature HTTP/1.1 HTTP/2
2813 // Multiplexing No (1 request per connection) Yes (multiple requests over a single connection)
```

```
2814 // Header compression No Yes (HPACK compression)
2815 // Server push ❌ ✅ (push resources proactively)
2816 // Latency Higher due to multiple connections Lower due to multiplexing and binary proto
2817
2818 // 2. How to implement Critical CSS:
2819
2820 // Extract CSS needed for above-the-fold content and inline it in <head> to render faster
2821
2822 // Example:
2823
2824 // <style>
2825 //   body { margin:0; font-family:sans-serif; }
2826 //   header { background:#333; color:#fff; }
2827 // </style>
2828 // <link rel="stylesheet" href="main.css">
2829
2830
2831 // Tools: Critical, Penthouse, or webpack plugins.
2832
2833 // 3. Efficient caching strategies for performance:
2834
2835 // Cache-Control: max-age, public/private, immutable for static assets.
2836
2837 // ETags: Track changes and reduce redundant downloads.
2838
2839 // Service workers: Offline caching and precaching.
2840
2841 // CDN caching: Serve resources from nearest edge servers.
2842
2843 // 4. Preventing layout shifts (CLS - Cumulative Layout Shift):
2844
2845 // Always set width/height for images and videos.
2846
2847 // Reserve space for dynamic content (ads, banners).
2848
2849 // Avoid inserting content above visible content after load.
2850
2851 // Example:
2852
2853 // 
2854
2855
2856 // 5. What is resource prefetching?
2857
2858 // Browser loads resources in advance for future use.
2859
2860 // Types:
2861
2862 // preload → needed now (critical resources).
2863
2864 // prefetch → needed later (next page or navigation).
2865
2866 // prerender → fully render future page in background.
```

```
2867
2868 // 6. Using Intersection Observer for lazy loading:
2869
2870 // Detects when elements enter the viewport and loads them dynamically.
2871
2872 // const observer = new IntersectionObserver((entries) => {
2873 //   entries.forEach(entry => {
2874 //     if(entry.isIntersecting) {
2875 //       entry.target.src = entry.target.dataset.src;
2876 //       observer.unobserve(entry.target);
2877 //     }
2878 //   });
2879 // });
2880 // document.querySelectorAll('img[data-src]').forEach(img => observer.observe(img));
2881
2882
2883 // 7. Code-splitting in React for performance:
2884
2885 // Split app into smaller chunks that load on demand using React.lazy or dynamic imports
2886
2887 // const LazyComp = React.lazy(() => import('./LazyComp'));
2888 // <Suspense fallback={<div>Loading...</div>}><LazyComp /></Suspense>
2889
2890
2891 // Reduces initial bundle size and improves load time.
2892
2893 // 8. Prioritizing critical resources for page load:
2894
2895 // Inline critical CSS.
2896
2897 // Use rel="preload" for fonts, scripts, and above-the-fold images.
2898
2899 // Defer non-critical JS with async or defer.
2900
2901 // 9. Optimizing performance of large image galleries:
2902
2903 // Use lazy loading (loading="lazy" or Intersection Observer).
2904
2905 // Serve responsive images (<picture> element or srcset).
2906
2907 // Compress images (WebP/AVIF).
2908
2909 // Consider virtualization to render only visible images.
2910
2911 // 10. What are Web Vitals and how to optimize Core Web Vitals?
2912
2913 // Web Vitals: Key metrics for UX performance:
2914
2915 // LCP (Largest Contentful Paint): Optimize server response, critical CSS, and images.
2916
2917 // FID (First Input Delay): Minimize JS execution, use web workers.
2918
```

```
2919 // CLS (Cumulative Layout Shift): Reserve space for images, ads, and dynamic content.
2920
2921
2922
2923
2924
2925
2926
2927
2928 // -----
2929
2930
2931 // Interview Question - Frontend Developer (React JS)
2932
2933 // I wanted to share some of the interesting and thought-provoking questions that were
2934
2935 // 1. Reverse a string without using reverse()
2936 // function reverseString(str) {
2937 //     let reversed = "";
2938 //     for (let i = str.length - 1; i >= 0; i--) {
2939 //         reversed += str[i];
2940 //     }
2941 //     return reversed;
2942 // }
2943
2944 // console.log(reverseString("Hello")); // Output: "olleH"
2945
2946 // 2. Generator Function in JavaScript
2947
2948 // A generator function allows you to pause execution and resume it later. It returns an
2949
2950 // function* generatorExample() {
2951 //     yield 1;
2952 //     yield 2;
2953 //     yield 3;
2954 // }
2955
2956 // const gen = generatorExample();
2957 // console.log(gen.next()); // { value: 1, done: false }
2958 // console.log(gen.next()); // { value: 2, done: false }
2959 // console.log(gen.next()); // { value: 3, done: false }
2960 // console.log(gen.next()); // { value: undefined, done: true }
2961
2962
2963 // * marks a generator.
2964
2965 // yield pauses the function and returns a value.
2966
2967 // 3. Closures in JavaScript
2968
2969 // A closure is a function that remembers the scope in which it was created.
2970
2971 // function outerFunction(outerVar) {
```

```

2971 // function outerFunction(outerVar) {
2972 //     return function innerFunction(innerVar) {
2973 //         console.log(`Outer: ${outerVar}, Inner: ${innerVar}`);
2974 //     };
2975 // }
2976
2977 // const closureExample = outerFunction("outside");
2978 // closureExample("inside"); // Outer: outside, Inner: inside
2979
2980
2981 // Inner function retains access to outerVar even after outerFunction finishes.
2982
2983 // 4. Higher-Order Components (HOC) in React
2984
2985 // A HOC is a function that takes a component and returns a new component with additional
2986
2987 // const withLogger = (WrappedComponent) => {
2988 //     return (props) => {
2989 //         console.log("Rendering component with props:", props);
2990 //         return <WrappedComponent {...props} />;
2991 //     };
2992 // };
2993
2994
2995 // Used for code reuse, logic abstraction, or conditional rendering.
2996
2997 // 5. Difference between State and Props
2998 // Aspect    State    Props
2999 // Mutability    Mutable (can be changed)    Immutable (read-only)
3000 // Scope    Local to the component    Passed from parent to child
3001 // Purpose    Manage internal component data    Pass data to child components
3002 // 6. Callback Function
3003
3004 // A callback is a function passed as an argument to another function, executed later.
3005
3006 // function greet(name, callback) {
3007 //     console.log(`Hello, ${name}`);
3008 //     callback();
3009 // }
3010
3011 // greet("Saikrishna", () => console.log("Callback executed!"));
3012
3013 // 7. Spread Operator ... in JavaScript
3014
3015 // Used to expand arrays/objects or merge them.
3016
3017 // // Array
3018 // const arr1 = [1, 2];
3019 // const arr2 = [...arr1, 3, 4];
3020 // console.log(arr2); // [1, 2, 3, 4]
3021
3022 // // Object
3023 // const obj1 = { a: 1, b: 2 };

```

```
3024 // const obj2 = { ...obj1, c: 3 };
3025 // console.log(obj2); // { a: 1, b: 2, c: 3 }
3026
3027 // 8. Shallow Copy vs Deep Copy
3028
3029 // Shallow Copy: Copies only first-level properties; nested objects still reference original
3030
3031 // Deep Copy: Copies everything recursively, creating independent nested objects.
3032
3033 // // Shallow copy
3034 // const obj1 = { a: 1, b: { c: 2 } };
3035 // const shallowCopy = { ...obj1 };
3036 // shallowCopy.b.c = 99;
3037 // console.log(obj1.b.c); // 99 (nested object still shared)
3038
3039 // // Deep copy
3040 // const deepCopy = JSON.parse(JSON.stringify(obj1));
3041 // deepCopy.b.c = 42;
3042 // console.log(obj1.b.c); // 99 (original remains unchanged)
3043
3044 // 9. Virtual DOM in React
3045
3046 // Virtual DOM is an in-memory representation of the real DOM.
3047
3048 // React updates only the necessary parts of the real DOM by comparing changes (diffing)
3049
3050 // 10. Hooks in React
3051
3052 // Hooks let you use state and lifecycle features in functional components.
3053
3054 // Common hooks:
3055
3056 // useState() → manage state
3057
3058 // useEffect() → handle side effects
3059
3060 // useContext() → access context
3061
3062 // useRef() → access DOM elements
3063
3064 // useMemo() / useCallback() → optimize performance
3065
3066 // import React, { useState, useEffect } from "react";
3067
3068 // function Counter() {
3069 //   const [count, setCount] = useState(0);
3070
3071 //   useEffect(() => {
3072 //     console.log(`Count updated: ${count}`);
3073 //   }, [count]);
3074
3075 //   return <button onClick={() => setCount(count + 1)}>Increment</button>;
3076 }
```

```
3076 // }
3077
3078
3079 // 11. useMemo vs React.memo
3080
3081 // useMemo: A hook that memoizes a computed value so it doesn't get recalculated on every render
3082
3083 // React.memo: A higher-order component that memoizes the whole functional component, preventing unnecessary re-renders
3084
3085 // import React, { useMemo } from "react";
3086
3087 // const ExpensiveComponent = ({ num }) => {
3088 //   const computed = useMemo(() => {
3089 //     console.log("Computing...");
3090 //     return num * 2;
3091 //   }, [num]); // recalculates only if num changes
3092
3093 //   return <div>{computed}</div>;
3094 // };
3095
3096 // export default React.memo(ExpensiveComponent); // memoizes component render
3097
3098
3099 // Difference:
3100
3101 // Feature   useMemo   React.memo
3102 // Scope     Value inside component   Entire component render
3103 // Usage     Prevent expensive recalcs   Prevent unnecessary re-renders
3104 // Dependency   Dependencies array required   Compares props by default
3105 // 12. Use cases for useCallback and useMemo
3106
3107 // useCallback: Memoizes a function reference to prevent re-creation on every render.
3108
3109 // useMemo: Memoizes a computed value.
3110
3111 // import React, { useState, useCallback, useMemo } from "react";
3112
3113 // function App() {
3114 //   const [count, setCount] = useState(0);
3115
3116 //   const double = useMemo(() => count * 2, [count]); // memoized value
3117
3118 //   const increment = useCallback(() => {
3119 //     setCount(prev => prev + 1);
3120 //   }, []); // memoized function
3121
3122 //   return (
3123 //     <div>
3124 //       <p>Double: {double}</p>
3125 //       <button onClick={increment}>Increment</button>
3126 //     </div>
3127 //   );
3128 // }
```

```
3129
3130
3131 // Use case: Optimize performance for components passed as props or expensive computatio
3132
3133 // 13. Timer with Start and Stop buttons
3134 // import React, { useState, useRef } from "react";
3135
3136 // function Timer() {
3137 //   const [time, setTime] = useState(0);
3138 //   const timerRef = useRef(null);
3139
3140 //   const startTimer = () => {
3141 //     if (!timerRef.current) {
3142 //       timerRef.current = setInterval(() => {
3143 //         setTime(prev => prev + 1);
3144 //       }, 1000);
3145 //     }
3146 //   };
3147
3148 //   const stopTimer = () => {
3149 //     clearInterval(timerRef.current);
3150 //     timerRef.current = null;
3151 //   };
3152
3153 //   return (
3154 //     <div>
3155 //       <h1>Time: {time}s</h1>
3156 //       <button onClick={startTimer}>Start</button>
3157 //       <button onClick={stopTimer}>Stop</button>
3158 //     </div>
3159 //   );
3160 // }
3161
3162 // 14. Handling lifecycle events in functional components
3163
3164 // Using useEffect:
3165
3166 // import React, { useEffect } from "react";
3167
3168 // function LifecycleDemo() {
3169 //   // componentDidMount
3170 //   useEffect(() => {
3171 //     console.log("Mounted");
3172 //
3173 //     // componentWillUnmount
3174 //     return () => console.log("Unmounted");
3175 //   }, []);
3176
3177 //   // componentDidUpdate (on count change)
3178 //   useEffect(() => {
3179 //     console.log("Updated");
3180 //   });
```

```
3181
3182 //   return <div>Check console for lifecycle logs</div>;
3183 // }
3184
3185 // 15. Passing data from child to parent
3186 // function Parent() {
3187 //   const handleData = (childData) => {
3188 //     console.log("Data from child:", childData);
3189 //   };
3190
3191 //   return <Child sendData={handleData} />;
3192 // }
3193
3194 // function Child({ sendData }) {
3195 //   return <button onClick={() => sendData("Hello Parent")}>Send</button>;
3196 // }
3197
3198
3199 // Pass a function from parent as prop and call it in the child.
3200
3201 // 16. Why copy state before updating in React
3202
3203 // React state must be immutable to detect changes and trigger re-renders.
3204
3205 // Direct mutation does not trigger re-render.
3206
3207 // const [arr, setArr] = useState([1, 2]);
3208 // const addItem = () => {
3209 //   setArr([...arr, 3]); // new array
3210 // };
3211
3212 // 17. Why use Redux in React
3213
3214 // Centralized state management
3215
3216 // Predictable state updates
3217
3218 // Easy debugging with DevTools
3219
3220 // Simplifies passing state across multiple components
3221
3222 // 18. Middlewares in Redux
3223
3224 // Functions that intercept actions before reaching the reducer
3225
3226 // Useful for:
3227
3228 // Logging
3229
3230 // Async requests
3231
3232 // Error handling
3233
```

```
3234 // const loggerMiddleware = (store) => (next) => (action) => {
3235 //   console.log("Dispatching:", action);
3236 //   return next(action);
3237 // };
3238
3239 // 19. redux-thunk vs redux-saga
3240
3241 // redux-thunk: Middleware to handle async logic using functions (simpler).
3242
3243 // redux-saga: Middleware to handle async using generator functions (more powerful, support
3244
3245 // redux-thunk example:
3246
3247 // const fetchUser = () => {
3248 //   return async (dispatch) => {
3249 //     dispatch({ type: "FETCH_USER_REQUEST" });
3250 //     try {
3251 //       const res = await fetch("/api/user");
3252 //       const data = await res.json();
3253 //       dispatch({ type: "FETCH_USER_SUCCESS", payload: data });
3254 //     } catch (err) {
3255 //       dispatch({ type: "FETCH_USER_FAILURE", error: err });
3256 //     }
3257 //   };
3258 // };
3259
3260 // 20. How reducers work and example
3261
3262 // Reducers are pure functions that take (state, action) and return a new state.
3263
3264 // const initialState = { count: 0 };
3265
3266 // function counterReducer(state = initialState, action) {
3267 //   switch (action.type) {
3268 //     case "INCREMENT":
3269 //       return { ...state, count: state.count + 1 };
3270 //     case "DECREMENT":
3271 //       return { ...state, count: state.count - 1 };
3272 //     default:
3273 //       return state;
3274 //   }
3275 // }
3276
3277
3278 // 21. Preventing component re-render in React
3279
3280 // Use React.memo to memoize functional components.
3281
3282 // Use useMemo / useCallback to memoize values or functions passed as props.
3283
3284 // const Child = React.memo(({ data }) => {
3285 //   console.log("Child rendered");
```

```
3286 //   return <div>{data}</div>;
3287 // });
3288
3289 // 22. Context API in React
3290
3291 // Provides a way to pass data through the component tree without props drilling.
3292
3293 // import React, { createContext, useContext } from "react";
3294
3295 // const ThemeContext = createContext();
3296
3297 // function Parent() {
3298 //   return (
3299 //     <ThemeContext.Provider value="dark">
3300 //       <Child />
3301 //     </ThemeContext.Provider>
3302 //   );
3303 // }
3304
3305 // function Child() {
3306 //   const theme = useContext(ThemeContext);
3307 //   return <div>Theme: {theme}</div>;
3308 // }
3309
3310 // 23. Sharing data across multiple components
3311
3312 // Options:
3313
3314 // Props drilling - simple but tedious for deep trees.
3315
3316 // Context API - avoid drilling.
3317
3318 // State management libraries - Redux, Zustand, Recoil.
3319
3320 // 24. React optimization techniques
3321
3322 // React.memo for functional components
3323
3324 // useMemo / useCallback for expensive computations and functions
3325
3326 // Code splitting and lazy loading
3327
3328 // Avoid anonymous functions in JSX
3329
3330 // Virtualization for long lists (e.g., react-window)
3331
3332 // 25. Lazy loading in React
3333
3334 // Loads components only when needed (improves performance).
3335
3336 // import React, { Suspense } from "react";
3337
```

```
3338 // const LazyComponent = React.lazy(() => import("../LazyComponent"));
3339
3340 // function App() {
3341 //   return (
3342 //     <Suspense fallback={<div>Loading...</div>}>
3343 //       <LazyComponent />
3344 //     </Suspense>
3345 //   );
3346 // }
3347
3348 // 26. Preventing unnecessary re-renders
3349
3350 // Use React.memo
3351
3352 // Use useMemo / useCallback
3353
3354 // Keep state local where possible
3355
3356 // Avoid changing references unnecessarily
3357
3358 // 27. Basic unit test for a component
3359
3360 // Using Jest + React Testing Library:
3361
3362 // import { render, screen, fireEvent } from "@testing-library/react";
3363 // import Counter from "../Counter";
3364
3365 // test("renders counter and increments", () => {
3366 //   render(<Counter />);
3367 //   const button = screen.getByText(/Increment/i);
3368 //   fireEvent.click(button);
3369 //   expect(screen.getByText(/Count: 1/i)).toBeInTheDocument();
3370 // });
3371
3372 // 28. Check if an object is empty in JS
3373 // const obj = {};
3374 // console.log(Object.keys(obj).length === 0); // true
3375
3376 // // OR
3377 // console.log(JSON.stringify(obj) === "{}"); // true
3378
3379 // 29. Bootstrap experience
3380
3381 // Usage: Rapid UI development using prebuilt components and responsive grid.
3382
3383 // Experience:
3384
3385 // Responsive layouts with container, row, col-*
3386
3387 // Buttons, modals, and navbars
3388
3389 // Integration with React (react-bootstrap)
3390
```

```
3391 // 30. Immediately Invoked Function Expression (IIFE)
3392
3393 // A function executed immediately after creation.
3394
3395 // Used for scoping variables and avoiding global pollution.
3396
3397 // (function() {
3398 //   const message = "Hello IIFE";
3399 //   console.log(message);
3400 // })();
3401
3402 // 31. Server-side rendering (SSR)
3403
3404 // Rendering React components on the server instead of the client.
3405
3406 // Benefits:
3407
3408 // Faster initial load
3409
3410 // SEO-friendly
3411
3412 // Experience: Using Next.js, pages are pre-rendered on the server and sent as HTML to c
3413
3414 // // Next.js example
3415 // export async function getServerSideProps() {
3416 //   const res = await fetch("https://api.example.com/data");
3417 //   const data = await res.json();
3418 //   return { props: { data } };
3419 // }
3420
3421 // 32. Implement a cricket score functionality in a table format that has 2 columns:
3422 // 1) Ball Number
3423 // 2) Number of runs
3424 // A button to update score
3425 import React, { useState } from "react";
3426
3427 ✓ function CricketScore() {
3428   const [balls, setBalls] = useState([]);
3429
3430 ✓   const addBall = () => {
3431     const run = Math.floor(Math.random() * 7); // Random runs between 0-6
3432     const ballNumber = balls.length + 1;
3433     setBalls([...balls, { ballNumber, run }]);
3434   };
3435
3436   return (
3437     <div style={{ padding: "20px", maxWidth: "400px", margin: "auto" }}>
3438       <h2>Cricket Score</h2>
3439       <table border="1" cellPadding="10" cellSpacing="0" width="100%">
3440         <thead>
3441           <tr>
3442             <th>Ball Number</th>
```

```

3443         <th>Runs</th>
3444     </tr>
3445 </thead>
3446 <tbody>
3447     {balls.map((ball) => (
3448         <tr key={ball.ballNumber}>
3449             <td>{ball.ballNumber}</td>
3450             <td>{ball.run}</td>
3451         </tr>
3452     ))}
3453 </tbody>
3454 </table>
3455 <button
3456     onClick={addBall}
3457     style={{ marginTop: "20px", padding: "10px 20px" }}
3458 >
3459     Add Ball
3460 </button>
3461 </div>
3462 );
3463 }
3464
3465 export default CricketScore;
3466
3467
3468 // -----
3469 // 1. What is Node.js?
3470
3471 // Node.js is a JavaScript runtime built on Chrome's V8 engine.
3472
3473 // It allows running JavaScript on the server-side.
3474
3475 // Key features:
3476
3477 // Non-blocking I/O
3478
3479 // Event-driven architecture
3480
3481 // Scalable for real-time applications
3482
3483 // # Example: Run a simple server
3484 // node server.js
3485
3486 // 2. Difference between Node.js and JavaScript
3487 // Feature   JavaScript   Node.js
3488 // Environment   Browser Server (V8 engine)
3489 // Usage         Client-side scripting   Server-side scripting
3490 // APIs           DOM, BOM       File system, HTTP, OS
3491 // Modules        No native module system   Built-in module system (CommonJS)
3492 // 3. Is Node.js single-threaded?
3493
3494 // Yes, Node.js runs on a single main thread.
3495

```

```
3496 // Uses event-driven, non-blocking I/O to handle multiple requests concurrently.
3497
3498 // Heavy computation can block the thread, so workers or child processes are recommended
3499
3500 // 4. What kind of API function is supported by Node.js?
3501
3502 // Node.js supports asynchronous and non-blocking API functions.
3503
3504 // Examples:
3505
3506 // File system: fs.readFile() (async) vs fs.readFileSync() (sync)
3507
3508 // HTTP server: http.createServer()
3509
3510 // const fs = require("fs");
3511 // fs.readFile("file.txt", "utf-8", (err, data) => {
3512 //   if (err) throw err;
3513 //   console.log(data);
3514 // });
3515
3516 // 5. What is a module in Node.js?
3517
3518 // A module is a reusable block of code in Node.js.
3519
3520 // Types:
3521
3522 // Built-in modules: fs, http, path
3523
3524 // User-defined modules: Your own JS files
3525
3526 // Third-party modules: Installed via npm
3527
3528 // // userModule.js
3529 // module.exports = function greet(name) { console.log(`Hello ${name}`); };
3530
3531 // // main.js
3532 // const greet = require("./userModule");
3533 // greet("Saikrishna"); // Hello Saikrishna
3534
3535 // 6. What is npm and its advantages?
3536
3537 // npm = Node Package Manager.
3538
3539 // Used to install, share, and manage packages for Node.js.
3540
3541 // Advantages:
3542
3543 // Easy dependency management
3544
3545 // Access to a huge library of packages
3546
3547 // Simplifies project setup
```

```
3548
3549 // npm install express
3550
3551 // 7. What is middleware?
3552
3553 // Middleware is a function that processes requests before sending a response.
3554
3555 // Commonly used in Express.js for logging, authentication, and parsing request data.
3556
3557 // const express = require("express");
3558 // const app = express();
3559
3560 // const logger = (req, res, next) => {
3561 //   console.log(`${req.method} ${req.url}`);
3562 //   next(); // move to next middleware
3563 // };
3564
3565 // app.use(logger);
3566
3567 // 8. How does Node.js handle concurrency despite being single-threaded?
3568
3569 // Node.js uses non-blocking I/O and an event loop.
3570
3571 // It delegates I/O tasks to the libuv thread pool and continues executing other tasks.
3572
3573 // Callback functions or Promises are invoked when the I/O task completes.
3574
3575 // 9. What is the control flow in Node.js?
3576
3577 // Node.js uses event-driven asynchronous control flow:
3578
3579 // Start execution
3580
3581 // Register callbacks for I/O or timers
3582
3583 // Continue execution
3584
3585 // Execute callbacks when events complete
3586
3587 // console.log("Start");
3588 // setTimeout(() => console.log("Middle"), 0);
3589 // console.log("End");
3590 // // Output: Start, End, Middle
3591
3592 // 10. What do you mean by the event loop in Node.js?
3593
3594 // The event loop allows Node.js to perform non-blocking I/O.
3595
3596 // It continuously checks:
3597
3598 // Callbacks queue
3599
```

```
3600 // Microtasks queue (Promises)
3601
3602 // Executes pending operations
3603
3604 // Flow: Event Loop → Callback Queue → Execution
3605 // 11. What are the main disadvantages of Node.js?
3606
3607 // Single-threaded nature can be a bottleneck for CPU-intensive tasks.
3608 // Callback hell can make code hard to read and maintain.
3609 // Limited support for multi-threading (though worker threads are available).
3610
3611 // 12. What is REPL in Node.js?
3612
3613 // REPL = Read-Eval-Print Loop.
3614
3615 // It is an interactive shell to execute Node.js commands line by line.
3616
3617 // Features:
3618
3619 // Read input
3620
3621 // Evaluate code
3622
3623 // Print output
3624
3625 // Loop back
3626
3627 // $ node
3628 // > console.log("Hello REPL")
3629 // Hello REPL
3630
3631 // 13. How to import a module in Node.js?
3632
3633 // Using CommonJS require():
3634
3635 // const fs = require("fs"); // built-in
3636 // const myModule = require("./myModule"); // custom module
3637
3638
3639 // Using ES Module import (requires "type": "module" in package.json):
3640
3641 // import fs from "fs";
3642 // import myModule from "./myModule.js";
3643
3644 // 14. Difference between Node.js and AJAX
3645 // Feature Node.js AJAX
3646 // Type Server-side runtime Client-side technique
3647 // Language JavaScript JS (XHR / Fetch API)
3648 // Purpose Build server apps Asynchronous requests from browser
3649 // Environment Server Browser
3650 // 15. What is package.json in Node.js?
3651
3652 // A manifest file containing:
```

```
3653
3654 // Project metadata (name, version, author)
3655
3656 // Dependencies
3657
3658 // Scripts (start, test, build)
3659
3660 // {
3661 //   "name": "my-app",
3662 //   "version": "1.0.0",
3663 //   "dependencies": {
3664 //     "express": "^4.18.2"
3665 //   },
3666 //   "scripts": {
3667 //     "start": "node index.js"
3668 //   }
3669 // }
3670
3671 // 16. Most popular Node.js framework
3672
3673 // Express.js - lightweight, flexible, and widely used for building APIs and web apps.
3674
3675 // 17. What are promises in Node.js?
3676
3677 // Promises represent asynchronous operations that may succeed or fail.
3678
3679 // States: pending, fulfilled, rejected.
3680
3681 // const fs = require("fs").promises;
3682
3683 // fs.readFile("file.txt", "utf8")
3684 //   .then((data) => console.log(data))
3685 //   .catch((err) => console.error(err));
3686
3687 // 18. Event-driven programming in Node.js
3688
3689 // Programming style where actions are triggered by events.
3690
3691 // Node.js emits events, and listeners handle them:
3692
3693 // const EventEmitter = require("events");
3694 // const emitter = new EventEmitter();
3695
3696 // emitter.on("message", (msg) => console.log(msg));
3697 // emitter.emit("message", "Hello Event!");
3698
3699 // 19. What is Buffer in Node.js?
3700
3701 // Buffer is used to handle binary data.
3702
3703 // Useful for reading files, streams, or network packets.
3704
```

```
3705 // const buffer = Buffer.from("Hello");
3706 // console.log(buffer); // <Buffer 48 65 6c 6c 6f>
3707
3708 // 20. What are streams in Node.js?
3709
3710 // Streams allow reading/writing data piece by piece rather than loading the whole file
3711
3712 // Types:
3713
3714 // Readable: fs.createReadStream()
3715
3716 // Writable: fs.createWriteStream()
3717
3718 // Duplex: both read and write
3719
3720 // Transform: modify data while reading/writing
3721
3722 // const fs = require("fs");
3723 // const readStream = fs.createReadStream("file.txt");
3724 // readStream.on("data", (chunk) => console.log(chunk.toString()));
3725
3726 // 21. Crypto module in Node.js
3727
3728 // Provides cryptographic functionalities:
3729
3730 // Hashing, encryption, decryption
3731
3732 // HMAC, signing, key generation
3733
3734 // const crypto = require("crypto");
3735 // const hash = crypto.createHash("sha256").update("hello").digest("hex");
3736 // console.log(hash);
3737
3738 // 22. Callback hell
3739
3740 // Nested callbacks causing hard-to-read code.
3741
3742 // doSomething((err, data1) => {
3743 //   if (err) return;
3744 //   doSomethingElse(data1, (err, data2) => {
3745 //     if (err) return;
3746 //     doAnotherThing(data2, (err, data3) => {
3747 //       console.log(data3);
3748 //     });
3749 //   });
3750 // });
3751
3752
3753 // Solutions: Promises, async/await
3754
3755 // 23. Timers module in Node.js
3756
3757 // Provides delayed or repeated execution.
```

```
3757 // provides delayed or repeated execution.
3758
3759 // setTimeout(callback, ms)
3760
3761 // setInterval(callback, ms)
3762
3763 // setImmediate(callback)
3764
3765 // 24. Difference between setImmediate() and process.nextTick()
3766 // Feature setImmediate() process.nextTick()
3767 // Execution time After I/O events in next loop Immediately after current operation
3768 // Purpose Schedule after I/O Schedule before next event loop
3769
3770
3771 // 25. Difference between setTimeout() and setImmediate()
3772 // Feature setTimeout() setImmediate()
3773 // Execution timing After specified delay After current poll phase of event loop
3774 // Use case Delay execution Execute immediately after I/O
3775
3776
3777 // 26. Difference between spawn() and fork()
3778 // Feature spawn() fork()
3779 // Purpose Launch new processes (any command) Launch new Node.js process
3780 // Communication Streams (stdout, stdin, stderr) IPC channel for sending messages
3781 // Use case Running shell commands Creating child Node processes
3782 // const { spawn, fork } = require("child_process");
3783
3784 // const ls = spawn("ls", ["-lh"]);
3785 // ls.stdout.on("data", (data) => console.log(`Output: ${data}`));
3786
3787 // const child = fork("child.js");
3788 // child.send({ msg: "Hello child" });
3789
3790 // 27. Use of Passport module
3791
3792 // Passport.js is a popular middleware for authentication in Node.js.
3793
3794 // Supports:
3795
3796 // Local username/password
3797
3798 // OAuth (Google, Facebook, GitHub)
3799
3800 // JWT authentication
3801
3802 // const passport = require("passport");
3803 // const LocalStrategy = require("passport-local").Strategy;
3804
3805 // passport.use(new LocalStrategy((username, password, done) => {
3806 // // Verify user
3807 // }));
3808
3809 // 28. What is a fork in Node.js?
```

```
3810
3811 // fork() creates a new Node.js child process with IPC (inter-process communication) ena
3812
3813 // Useful for scaling apps or offloading CPU-heavy tasks.
3814
3815 // 29. Three methods to avoid callback hell
3816
3817 // Promises
3818
3819 // Async/Await
3820
3821 // Modularizing callbacks into separate functions
3822
3823 // async function fetchData() {
3824 //   const data1 = await getData1();
3825 //   const data2 = await getData2(data1);
3826 //   console.log(data2);
3827 // }
3828
3829 // 30. What is body-parser?
3830
3831 // Middleware to parse incoming request bodies in Express.js.
3832
3833 // Converts JSON or URL-encoded data to req.body.
3834
3835 // const bodyParser = require("body-parser");
3836 // app.use(bodyParser.json());
3837 // app.use(bodyParser.urlencoded({ extended: true }));
3838
3839 // 31. What is CORS?
3840
3841 // Cross-Origin Resource Sharing - allows controlled access to resources from different
3842
3843 // Enable in Express:
3844
3845 // const cors = require("cors");
3846 // app.use(cors({ origin: "*" }));
3847
3848 // 32. TLS module in Node.js
3849
3850 // Provides Transport Layer Security for secure communication.
3851
3852 // Can create HTTPS servers or clients.
3853
3854 // const tls = require("tls");
3855 // const server = tls.createServer(options, (socket) => {
3856 //   console.log("Secure connection established");
3857 // });
3858
3859 // 33. What is a cluster in Node.js?
3860
3861 // Clusters allow running multiple Node.js processes to utilize multi-core CPUs.
```

```
3862
3863 // Improves performance and handles more requests.
3864
3865 // const cluster = require("cluster");
3866 // const http = require("http");
3867 // const numCPUs = require("os").cpus().length;
3868
3869 // if (cluster.isMaster) {
3870 //   for (let i = 0; i < numCPUs; i++) cluster.fork();
3871 // } else {
3872 //   http.createServer((req, res) => res.end("Hello")).listen(8000);
3873 // }
3874
3875 // 34. Managing sessions in Node.js
3876
3877 // Use express-session middleware:
3878
3879 // const session = require("express-session");
3880
3881 // app.use(session({
3882 //   secret: "mySecret",
3883 //   resave: false,
3884 //   saveUninitialized: true,
3885 //   cookie: { maxAge: 60000 }
3886 // }));
3887
3888 // 35. Types of streams in Node.js
3889
3890 // Readable - read data (fs.createReadStream)
3891
3892 // Writable - write data (fs.createWriteStream)
3893
3894 // Duplex - read & write (net.Socket)
3895
3896 // Transform - modify data while streaming (zlib.createGzip())
3897
3898 // 36. Implementing authentication and authorization
3899
3900 // Authentication: Verify user identity (e.g., Passport.js, JWT).
3901
3902 // Authorization: Control access based on roles/permissions.
3903
3904 // // Example: JWT
3905 // const jwt = require("jsonwebtoken");
3906 // app.post("/login", (req, res) => {
3907 //   const token = jwt.sign({ userId: 1 }, "secret");
3908 //   res.json({ token });
3909 // });
3910
3911 // 37. Packages for file uploading
3912
3913 // multer - most popular middleware for handling file uploads in Express.js.
3914
```

```
3915 // const multer = require("multer");
3916 // const upload = multer({ dest: "uploads/" });
3917 // app.post("/upload", upload.single("file"), (req, res) => {
3918 //   res.send("File uploaded");
3919 // });
3920
3921 // 38. Handling database connections
3922
3923 // Use database drivers or ORMs (Mongoose for MongoDB, Sequelize for SQL).
3924
3925 // Example (MongoDB + Mongoose):
3926
3927 // const mongoose = require("mongoose");
3928 // mongoose.connect("mongodb://localhost/mydb")
3929 //   .then(() => console.log("DB Connected"))
3930 //   .catch(err => console.error(err));
3931
3932 // 39. Reading command line arguments
3933 // // Run: node app.js arg1 arg2
3934 // console.log(process.argv); // ['node', 'app.js', 'arg1', 'arg2']
3935
3936 // 40. What are child processes?
3937
3938 // Child processes allow running external commands or scripts from Node.js.
3939
3940 // Methods: spawn(), fork(), exec(), execFile().
3941
3942 // const { exec } = require("child_process");
3943 // exec("ls", (err, stdout) => console.log(stdout));
3944 // -----
3945
3946 // **🚀 Level Up Your JavaScript Skills with These 40 Essential Interview Questions! 🧑‍💻
3947
3948 // 1. 💡 What are the different data types in JavaScript?
3949 // 2. 📈 What is hoisting in JavaScript?
3950 // 3. ❓ What is the difference between `null` and `undefined`?
3951 // 4. 🔒 What are closures in JavaScript?
3952 // 5. 🔄 What is a callback function?
3953 // 6. ⚙️ What are promises in JavaScript?
3954 // 7. ⌚ What is the purpose of the `setTimeout()` function?
3955 // 8. 🔍 How can you check if an array includes a certain value?
3956 // 9. 🗑️ How can you remove duplicates in an array?
3957 // 10. 🔄 What is the purpose of `async` and `await` in JavaScript?
3958 // 11. ❓ What is the difference between `==` and `===`?
3959 // 12. 🔄 What is the difference between `let`, `const`, and `var`?
3960 // 13. 🔄 What is the difference between `for`, `while`, and `do-while` loops?
3961 // 14. 📍 What is the difference between `this` in the global scope and `this` in a fun
3962 // 15. ❓ What is the difference between `null` and `NaN`?
3963 // 16. ❓ What is the difference between `undefined` and `undeclared`?
3964 // 17. 🧱 What is the difference between `Object` and `Array`?
3965 // 18. 🔄 What is the difference between `map`, `filter`, and `reduce`?
3966 // 19. 🔍 What is the difference between `instanceof` and `typeof`?
```

```
3967 // 20. 📏 What is the difference between `throw` and `try-catch`?
3968 // 21. 📏 What is the difference between `JSON.stringify()` and `JSON.parse()`?
3969 // 22. 📏 What is the difference between `Array.from()` and `Array.of()`?
3970 // 23. 📏 What is the difference between `Object.assign()` and the spread operator?
3971 // 24. 📏 What is the difference between `Array.push()` and `Array.unshift()`?
3972 // 25. 📏 What is the difference between `Array.pop()` and `Array.shift()`?
3973 // 26. ✂️ What is the difference between `Array.slice()` and `Array.splice()`?
3974 // 27. 📏 What is the difference between `Array.concat()` and the spread operator?
3975 // 28. 📏 What is the difference between `Array.every()` and `Array.some()`?
3976 // 29. 🔍 What is the difference between `Array.find()` and `Array.findIndex()`?
3977 // 30. 🔍 What is the difference between `Array.includes()` and `Array.indexOf()`?
3978 // 31. 📏 What is the difference between `Array.sort()` and `Array.reverse()`?
3979 // 32. 🔗 What is the difference between `Array.join()` and `Array.toString()`?
3980 // 33. 📏 What is the difference between `Array.length` and `Object.keys().length`?
3981 // 34. 🔑 What is the difference between `Object.keys()` and `Object.values()`?
3982 // 35. 📏 What is the difference between `Object.freeze()` and `Object.seal()`?
3983 // 36. 📏 What is the difference between `Object.create()` and `Object.assign()`?
3984 // 37. 📏 What is the difference between `Object.getPrototypeOf()` and `Object.setPrototypeOf`?
3985 // 38. 🔑 What is the difference between `Object.keys()` and `Object.getOwnPropertyNames`?
3986 // 39. 📏 What is the difference between `Object.entries()` and `Object.fromEntries()`?
3987 // 40. 🔑 What is the difference between `Object.values()` and `Object.entries()`?
3988
3989 // -----
3990 // Commonly asked Frontend interview questions.
3991
3992 // ➤ Performance – Speed, efficiency, and user experience
3993 // 1. How do you lazy-load images or components?
3994 // 2. What are critical rendering path and render-blocking resources?
3995 // 3. What is tree-shaking and why does it matter in bundlers like Webpack?
3996 // 4. How do you reduce reflows and repaints in the browser?
3997 // 5. How do you implement debouncing and throttling in JS?
3998 // 6. What causes Cumulative Layout Shift (CLS), and how do you fix it?
3999 // 7. How do you optimize font loading?
4000 // 8. How do you analyze performance using Chrome DevTools?
4001 // 9. What's the impact of large DOM trees?
4002 // 10. How do you use code-splitting in large React apps?
4003
4004 // ➤ Accessibility (a11y)
4005 // 1. What are ARIA roles and when should you use them?
4006 // 2. How do you manage keyboard navigation for custom components?
4007 // 3. What's the role of tabindex in focus management?
4008 // 4. How do you test accessibility in your applications?
4009 // 5. What's the difference between live regions and alerts?
4010 // 6. How can headings impact screen reader navigation?
4011 // 7. What is color contrast and why does it matter?
4012 // 8. How do labels and placeholders affect screen reader users?
4013 // 9. What are some tools used for accessibility testing (NVDA, axe, Lighthouse)?
4014 // 10. How do you ensure a modal is accessible?
4015
4016 // ➤ JavaScript
4017 // 1. What is the Document Object Model (DOM), and how does JavaScript interact with it?
4018 // 2. What are closures in JavaScript, and how do they work?
4019 // 3. What is the this keyword in JavaScript, and how does it behave in different contexts?
```

```
4020 // 4. What is a JavaScript promise, and how does it handle asynchronous code?
4021 // 5. What is the event loop, and how does JavaScript handle asynchronous operations?
4022 // 6. What is hoisting in JavaScript, and how does it work?
4023 // 7. What are JavaScript data types, and how do you check the type of a variable?
4024 // 8. What is the difference between null and undefined in JavaScript?
4025 // 9. What is a callback function, and how is it used?
4026 // 10. What is bind(), call(), and apply() in JavaScript, and when do you use them?
4027 // 11. What is the difference between setTimeout() and setInterval()?
4028 // 12. How do JavaScript promises work, and what is the then() method?
4029 // 13. What is async/await, and how does it simplify asynchronous code in JavaScript?
4030 // 14. What are the advantages of using async functions over callbacks?
4031 // 15. What are JavaScript modules, and how do you import/export them?
4032 // 16. What are higher-order functions in JavaScript, and can you provide an example?
4033 // 17. What is destructuring in JavaScript, and how is it useful?
4034 // 18. What are template literals in JavaScript, and how do they work?
4035 // 19. How does the spread operator work in JavaScript?
4036 // 20. What is the rest parameter in JavaScript, and how does it differ from the argument
4037 // 21. How do you prevent default actions and stop event propagation in JavaScript?
4038 // 22. What is the prototype chain in JavaScript, and how does inheritance work?
4039 // 23. How do you clone an object or array in JavaScript?
4040
4041 // -----
4042
4043 // Node.js Interview Preparation hashtag#part7 !!
4044
4045 // What is Express.js?
4046 // Express.js is a minimal and flexible Node.js web application framework that simplifies
4047
4048 // What are the main features of Express.js?
4049 // Routing
4050 // Middleware support
4051 // Template engines integration
4052 // HTTP utility methods
4053 // Quick API creation
4054
4055 // How do you create a basic Express.js server?
4056 // You create a basic Express server by requiring the express module, creating an app in
4057
4058 // What is middleware in Express.js?
4059 // Middleware in Express.js refers to functions that have access to the request, response
4060
4061 // How do you handle route parameters in Express.js?
4062 // Route parameters are used to capture values in the URL path. You can access them via
4063
4064 // What is the role of next() in Express.js?
4065 // The next() function is used to pass control to the next middleware function in the stack
4066
4067 // How do you serve static files in Express.js?
4068 // You can serve static files using express.static() middleware, which specifies the folder
4069
4070 // How do you handle POST data in Express.js?
4071 // Use express.json() or express.urlencoded() middleware to parse JSON or URL-encoded data
```

4072

4073 // What is routing in Express.js?

4074 // Routing refers to defining how the application should respond to client requests to s